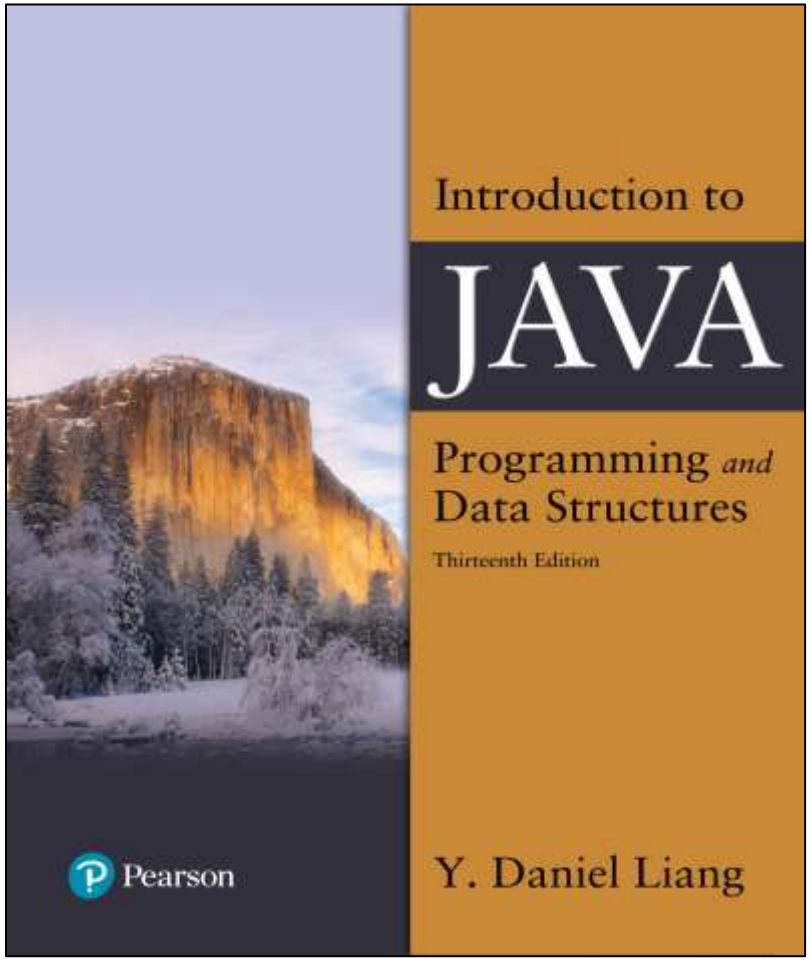
Introduction to Java Programming and Data Structures

Thirteenth Edition



Chapter 6

Methods

Pearson

Copyright © 2024 Pearson Education, Inc. All Rights Reserved

Opening Problem

Find the sum of integers from 1 to 10, from 20 to 30, and from 35 to 45, respectively.

Pearson

Copyright © 2024 Pearson Education, Inc. All Rights Reserved

Problem (1 of 2)

```
int sum = 0;
for (int i = 1; i <= 10; i++)
  sum += i;
System.out.println("Sum from 1 to 10 is " + sum);

sum = 0;
for (int i = 20; i <= 30; i++)
  sum += i;
System.out.println("Sum from 20 to 30 is " + sum);

sum = 0;
for (int i = 35; i <= 45; i++)
  sum += i;
System.out.println("Sum from 35 to 45 is " + sum);
```

Pearson

Copyright © 2024 Pearson Education, Inc. All Rights Reserved

Problem (2 of 2)

```
int sum = 0;
for (int i = 1; i <= 10; i++)
  sum += i;
System.out.println("Sum from 1 to 10 is " + sum);

sum = 0;
for (int i = 20; i <= 30; i++)
  sum += i;
System.out.println("Sum from 20 to 30 is " + sum);

sum = 0;
for (int i = 35; i <= 45; i++)
  sum += i;
System.out.println("Sum from 35 to 45 is " + sum);
```

Pearson

Copyright © 2024 Pearson Education, Inc. All Rights Reserved

Solution

```
public static int sum(int i1, int i2) {
  int sum = 0;
  for (int i = i1; i <= i2; i++)
    sum += i;
  return sum;
}
```

```
public static void main(String[] args) {
  System.out.println("Sum from 1 to 10 is " + sum(1, 10));
  System.out.println("Sum from 20 to 30 is " + sum(20, 30));
  System.out.println("Sum from 35 to 45 is " + sum(35, 45));
}
```

MethodDemo

Pearson

Copyright © 2024 Pearson Education, Inc. All Rights Reserved

Objectives (1 of 2)

6.1 To define methods with formal parameters (§6.2).

6.2 To invoke methods with actual parameters (i.e., arguments) (§6.2).

6.3 To define methods with a return value (§6.3).

6.4 To define methods without a return value (§6.4).

6.5 To pass arguments by value (§6.5).

6.6 To develop reusable code that is modular, easy to read, easy to debug, and easy to maintain (§6.6).

6.7 To write a method that converts hexadecimals to decimals (§6.7).

Pearson

Copyright © 2024 Pearson Education, Inc. All Rights Reserved

Objectives (2 of 2)

6.8 To use method overloading and understand ambiguous overloading (§6.8).

6.9 To determine the scope of variables (§6.9).

6.10 To apply the concept of method abstraction in software development (§6.10).

6.11 To design and implement methods using stepwise refinement (§6.11).

Pearson

Copyright © 2024 Pearson Education, Inc. All Rights Reserved

Defining Methods (1 of 2)

A method is a collection of statements that are grouped together to perform an operation.

Define a method

```
public static int max(int num1, int num2) {

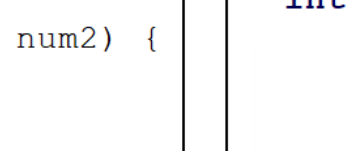
   int result;

   if (num1 > num2)
     result = num1;
   else
     result = num2;

   return result;
}
```

Invoke a method

```
int z = max(x, y);
```

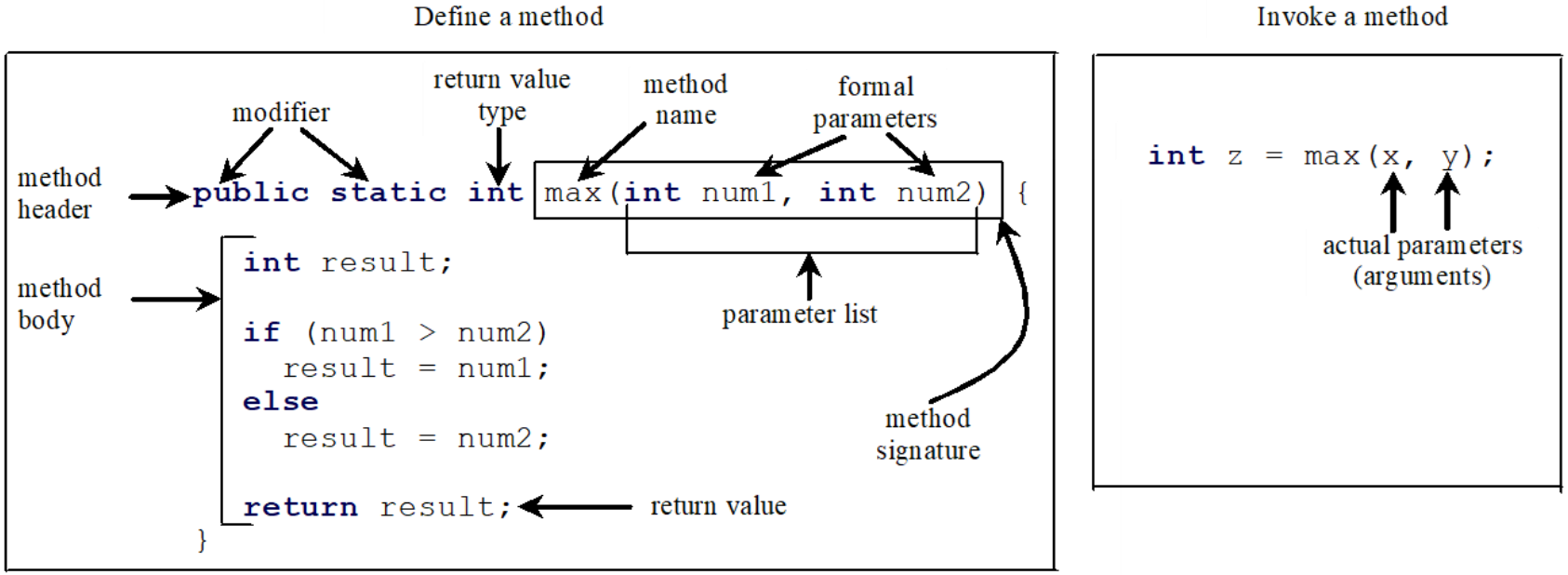


Pearson

Copyright © 2024 Pearson Education, Inc. All Rights Reserved

Defining Methods (2 of 2)

A method is a collection of statements that are grouped together to perform an operation.

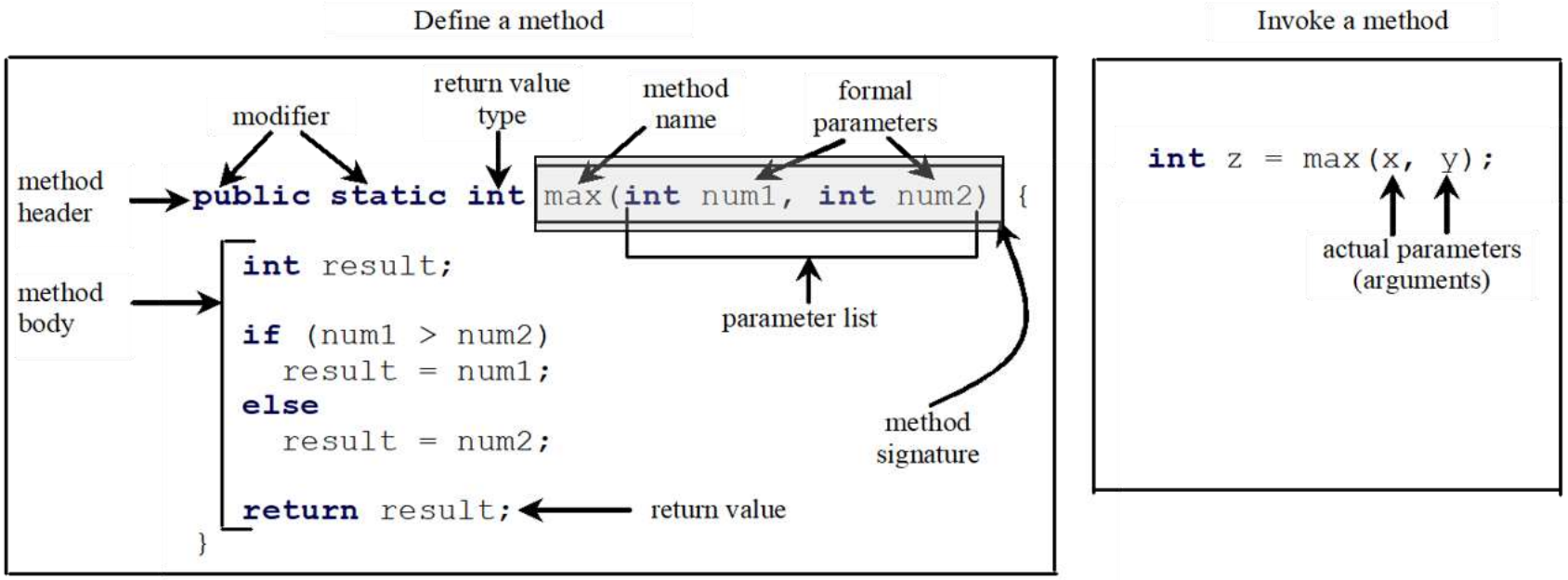


Pearson

Copyright © 2024 Pearson Education, Inc. All Rights Reserved

Method Signature

Method signature is the combination of the method name and the parameter list.

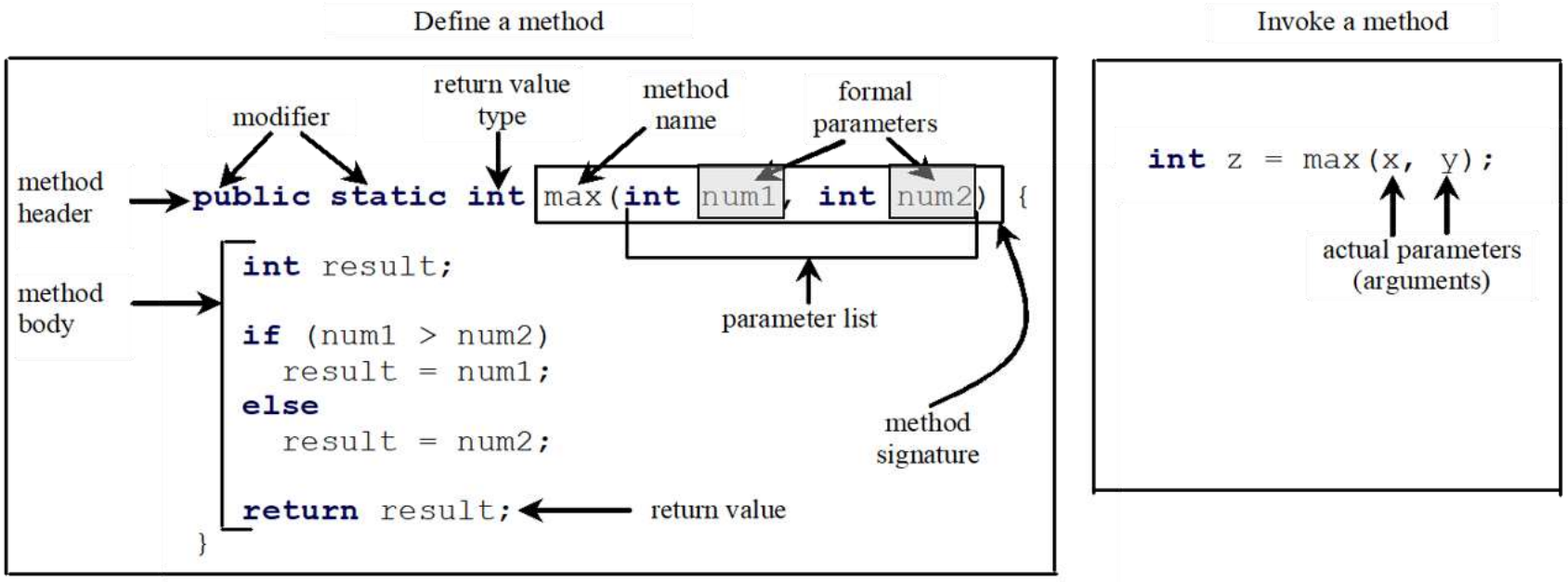


Pearson

Copyright © 2024 Pearson Education, Inc. All Rights Reserved

Formal Parameters

The variables defined in the method header are known as **formal parameters**.

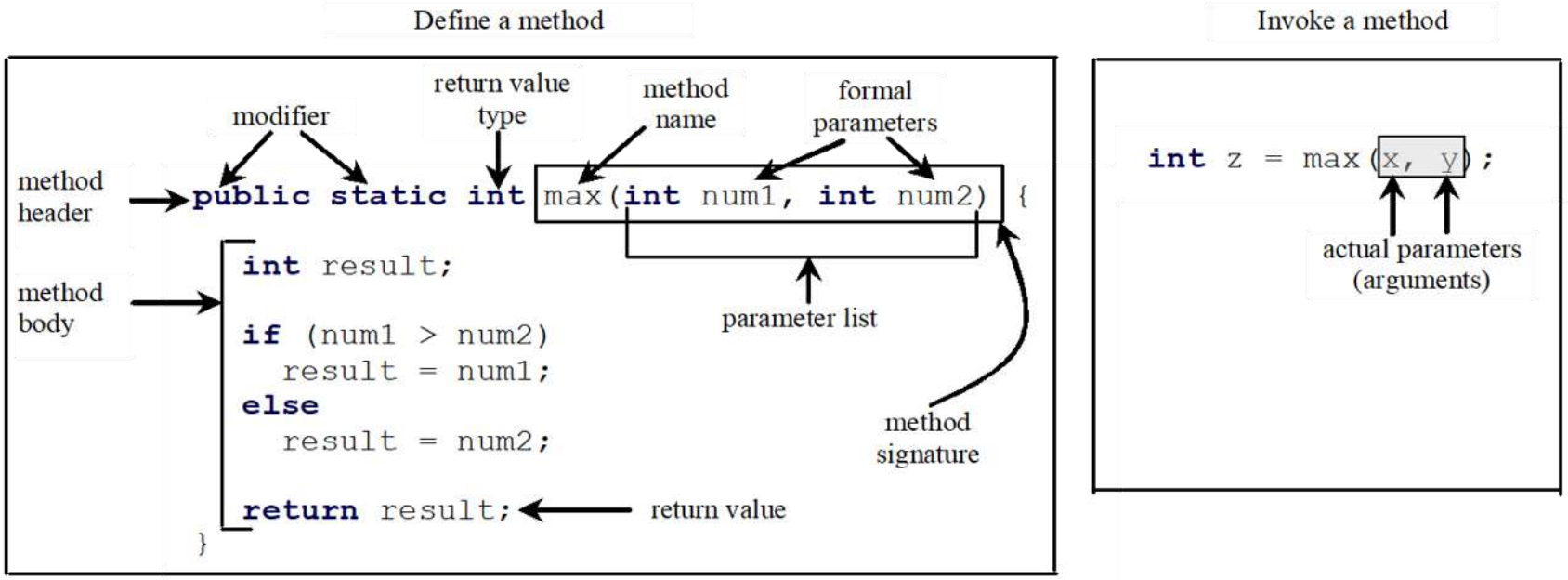


Pearson

Copyright © 2024 Pearson Education, Inc. All Rights Reserved

Actual Parameters

When a method is invoked, you pass a value to the parameter. This value is referred to as **actual parameter or argument**.

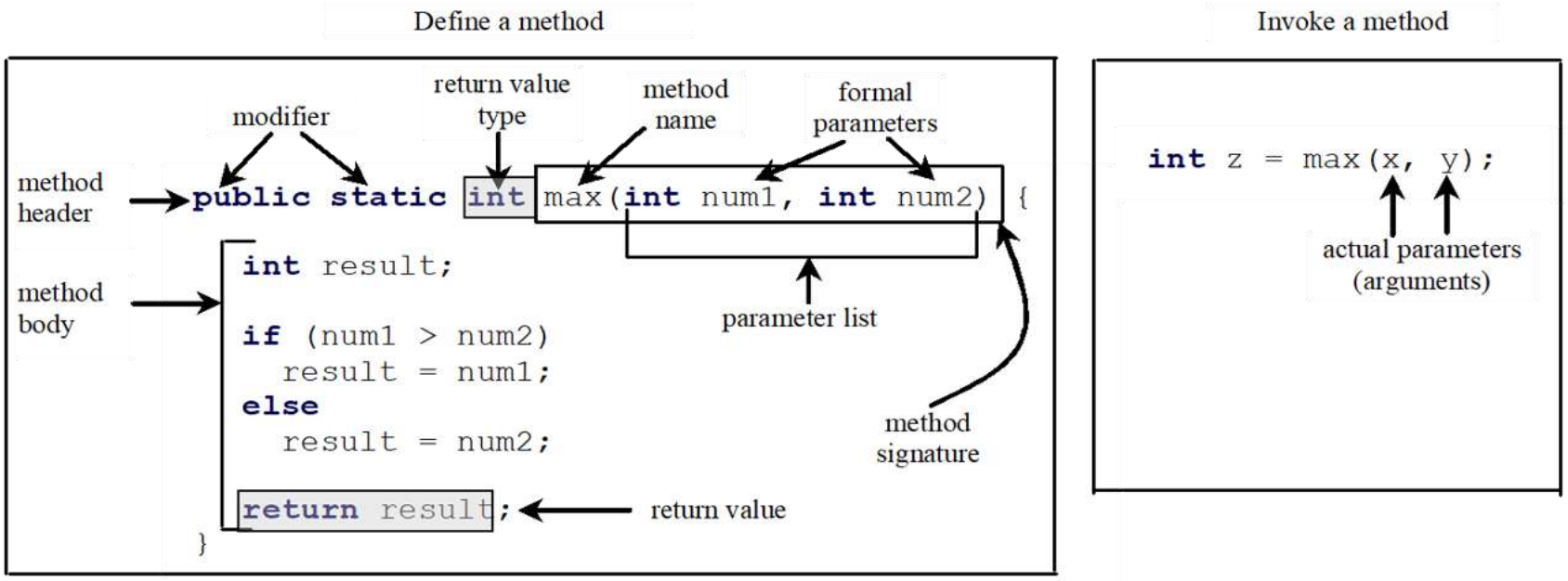


Pearson

Copyright © 2024 Pearson Education, Inc. All Rights Reserved

Return Value Type

A method may return a value. The **`returnValueType`** is the data type of the value the method returns. If the method does not return a value, the **`returnValueType`** is the keyword **`void`**. For example, the **`returnValueType`** in the **`main`** method is **`void`**.



Pearson

Copyright © 2024 Pearson Education, Inc. All Rights Reserved

Calling Methods (1 of 2)

Testing the `max` method

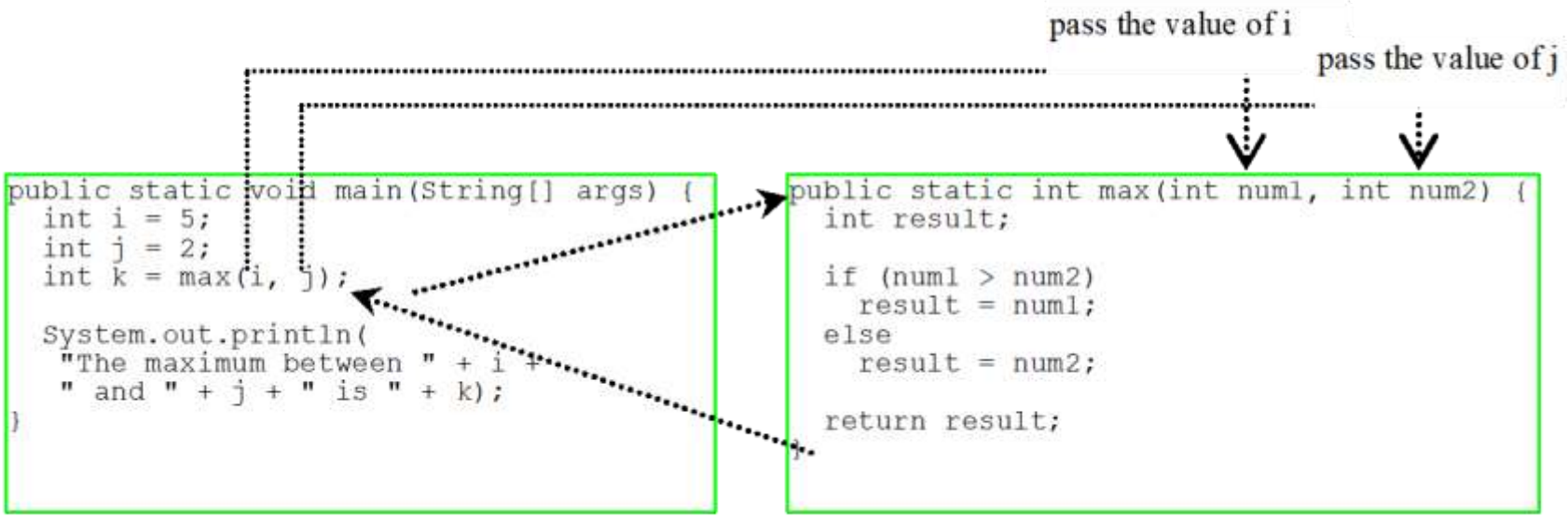
This program demonstrates calling a method `max` to return the largest of the `int` values

TestMax

Pearson

Copyright © 2024 Pearson Education, Inc. All Rights Reserved

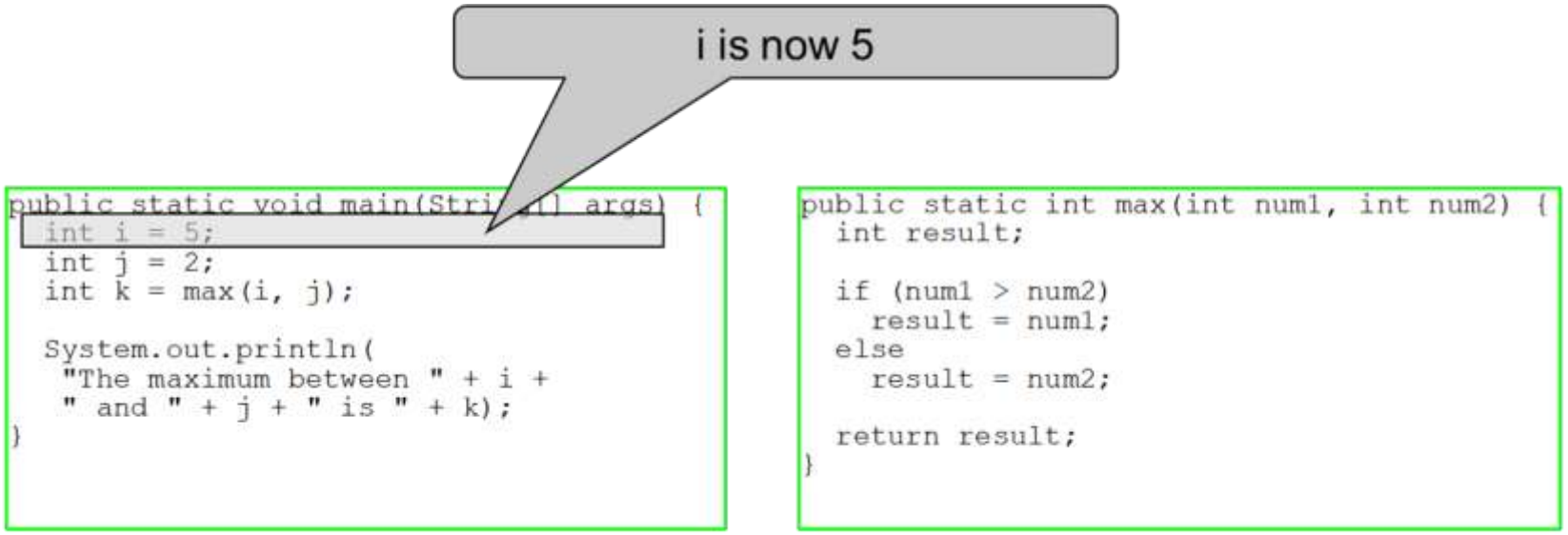
Calling Methods (2 of 2)



Pearson

Copyright © 2024 Pearson Education, Inc. All Rights Reserved

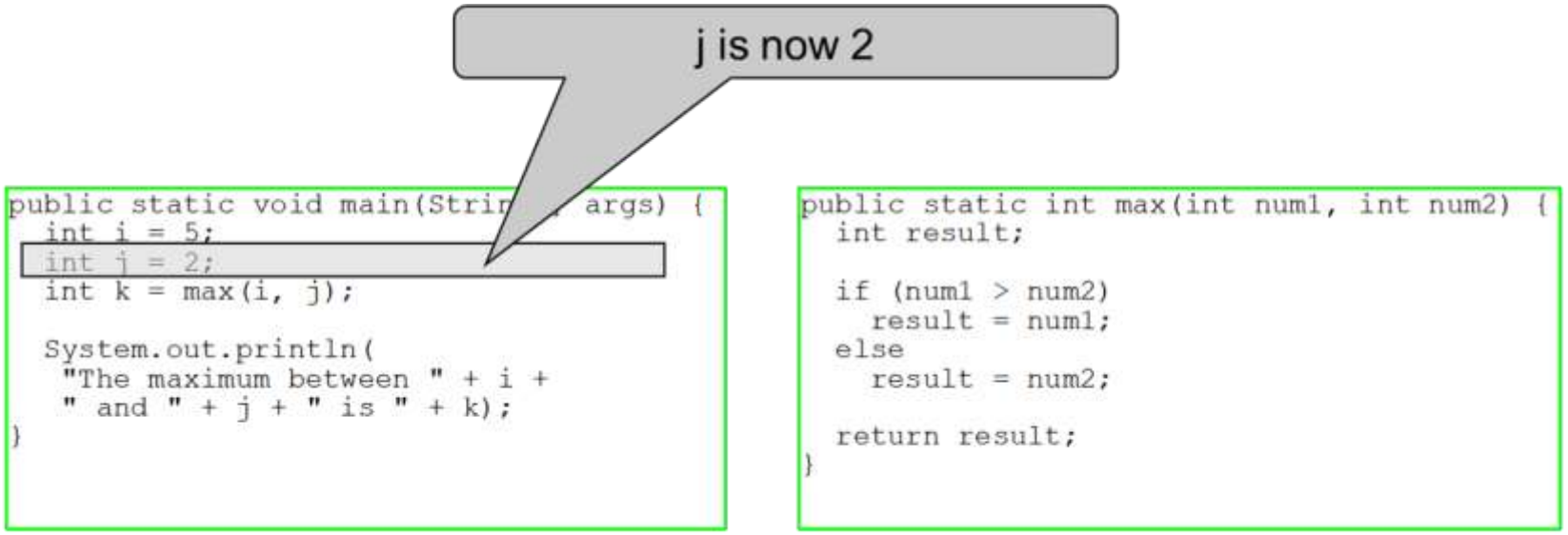
Trace Method Invocation (1 of 10)



Pearson

Copyright © 2024 Pearson Education, Inc. All Rights Reserved

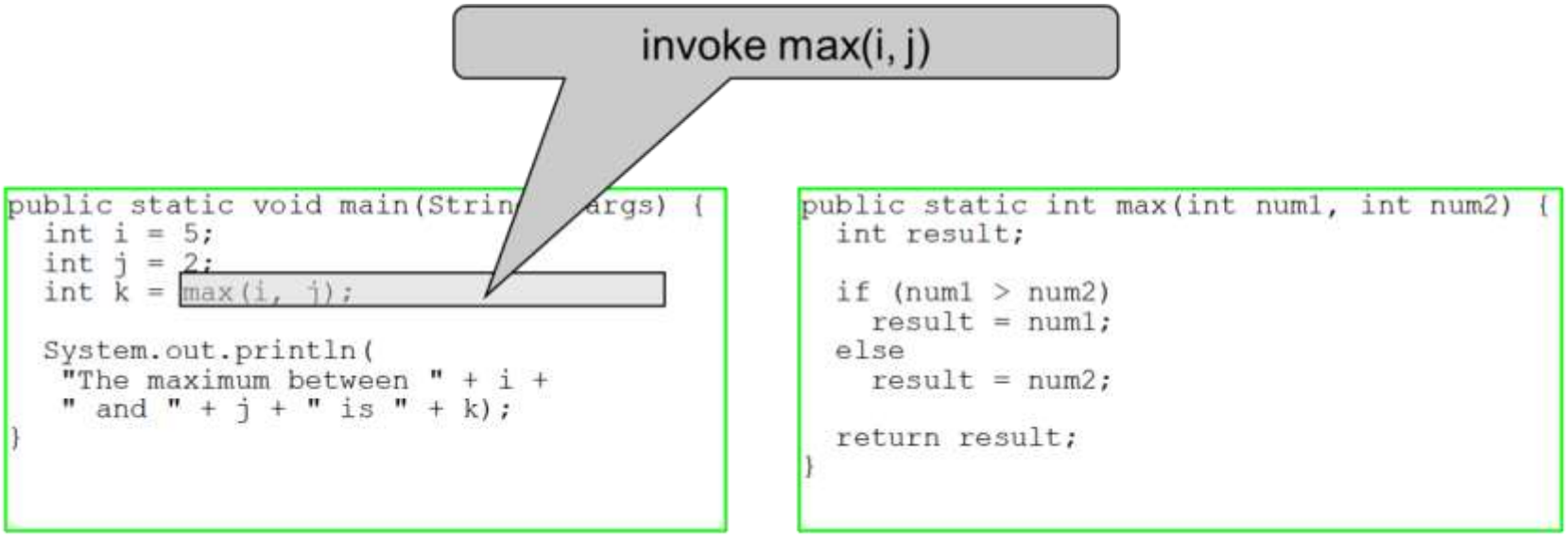
Trace Method Invocation (2 of 10)



Pearson

Copyright © 2024 Pearson Education, Inc. All Rights Reserved

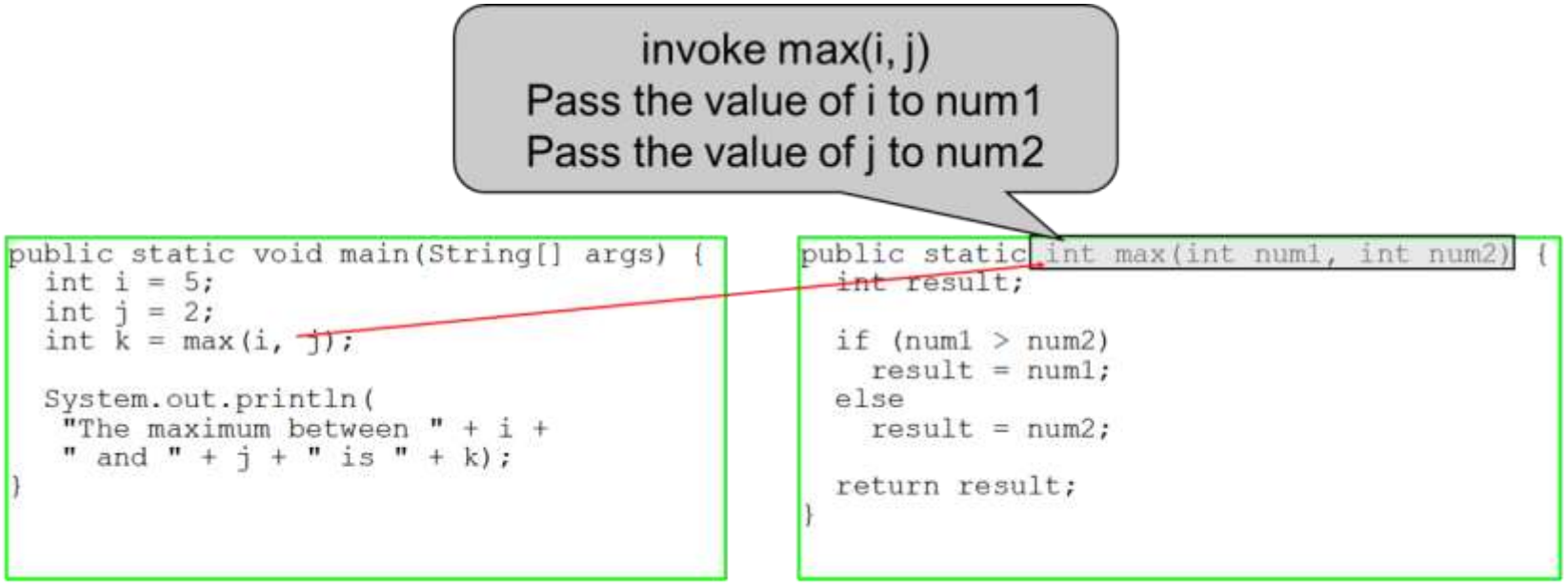
Trace Method Invocation (3 of 10)



Pearson

Copyright © 2024 Pearson Education, Inc. All Rights Reserved

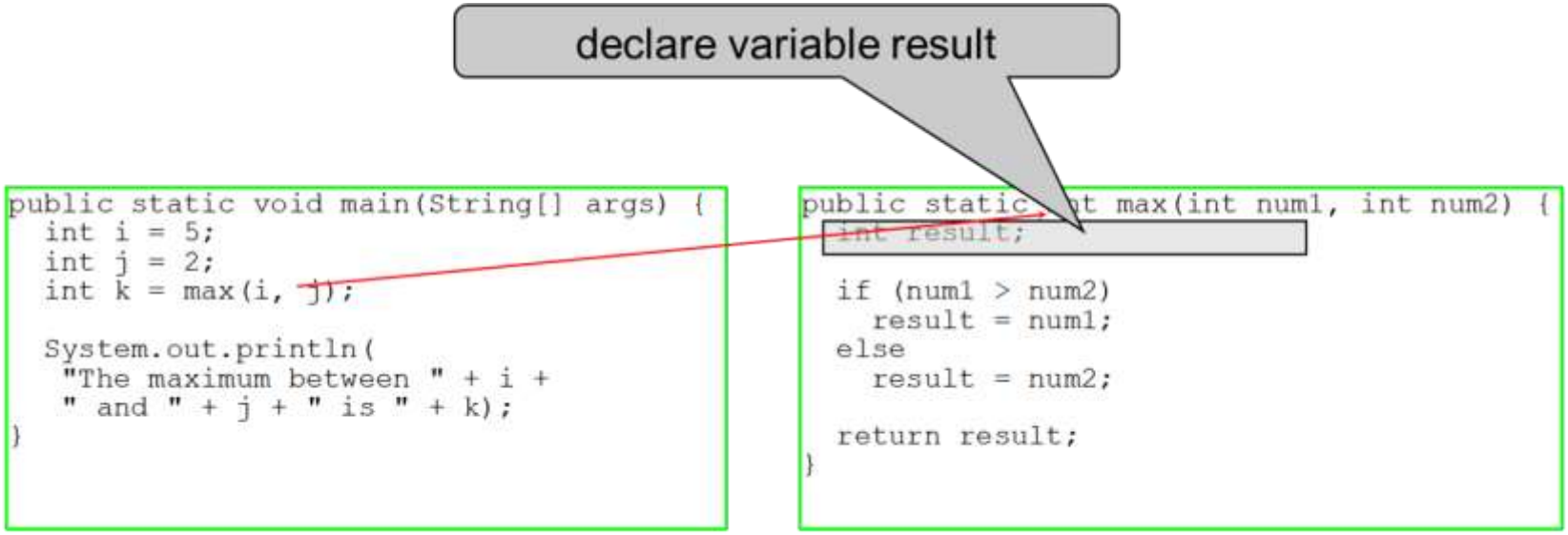
Trace Method Invocation (4 of 10)



Pearson

Copyright © 2024 Pearson Education, Inc. All Rights Reserved

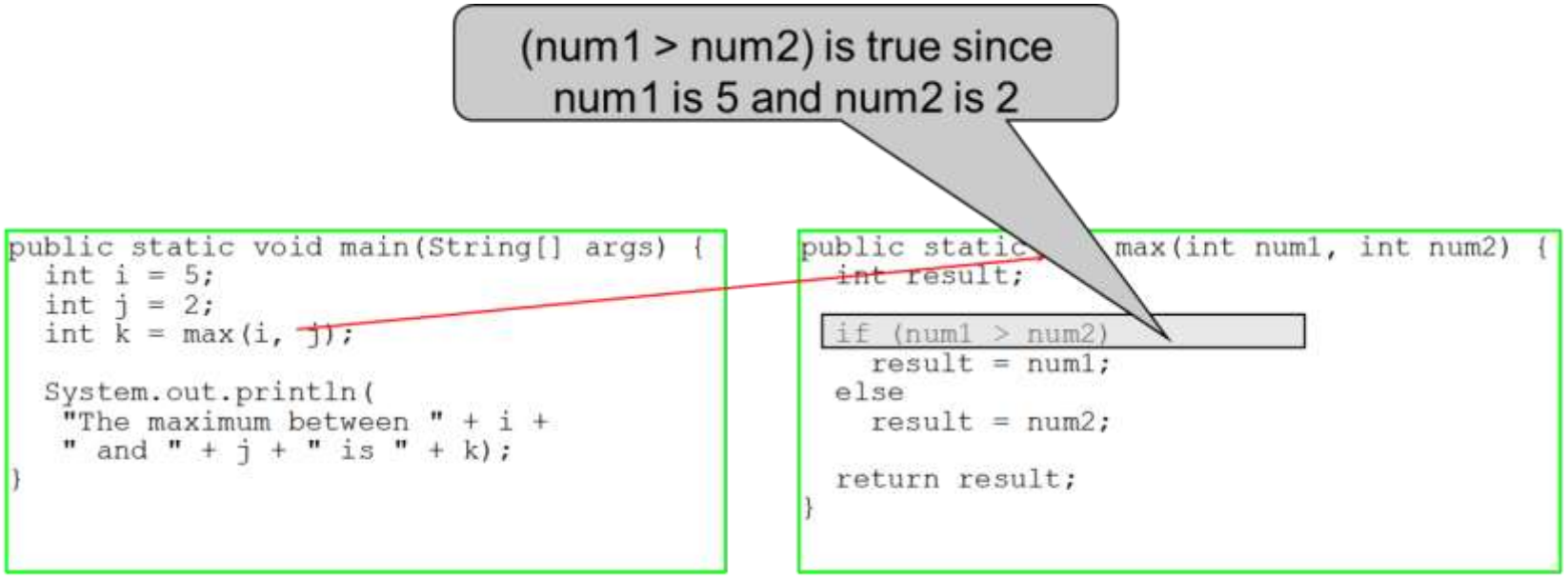
Trace Method Invocation (5 of 10)



Pearson

Copyright © 2024 Pearson Education, Inc. All Rights Reserved

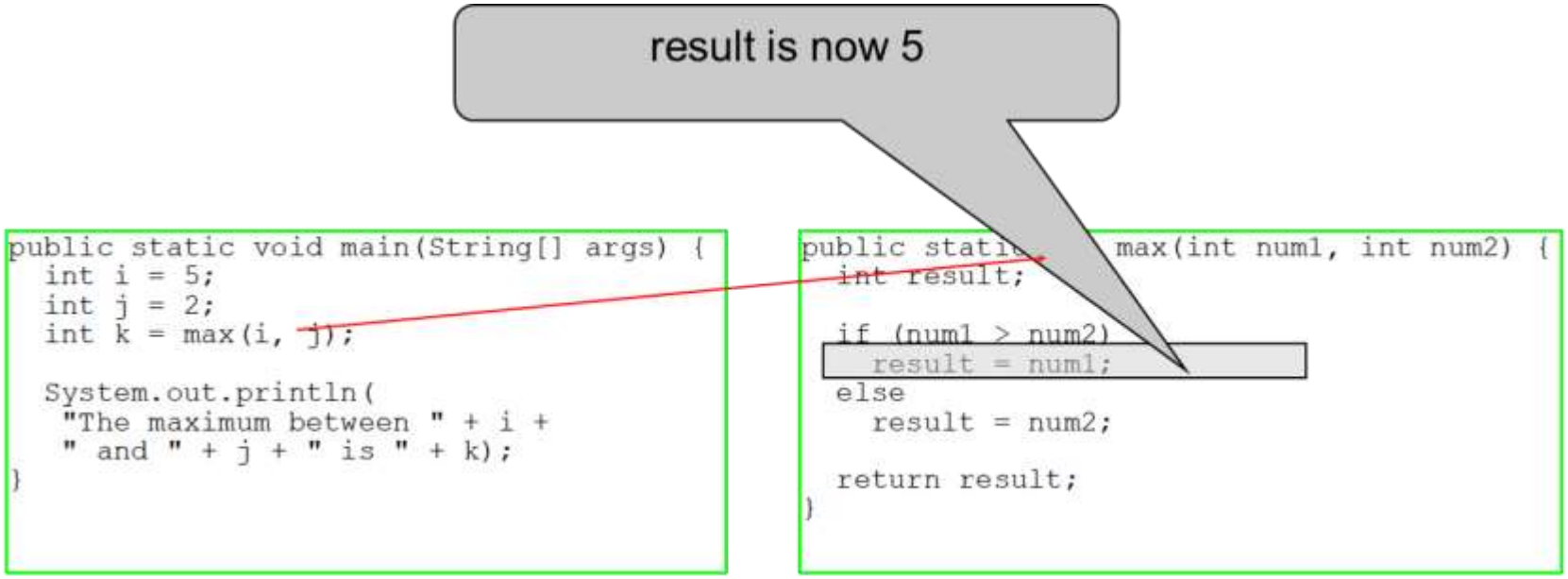
Trace Method Invocation (6 of 10)



Pearson

Copyright © 2024 Pearson Education, Inc. All Rights Reserved

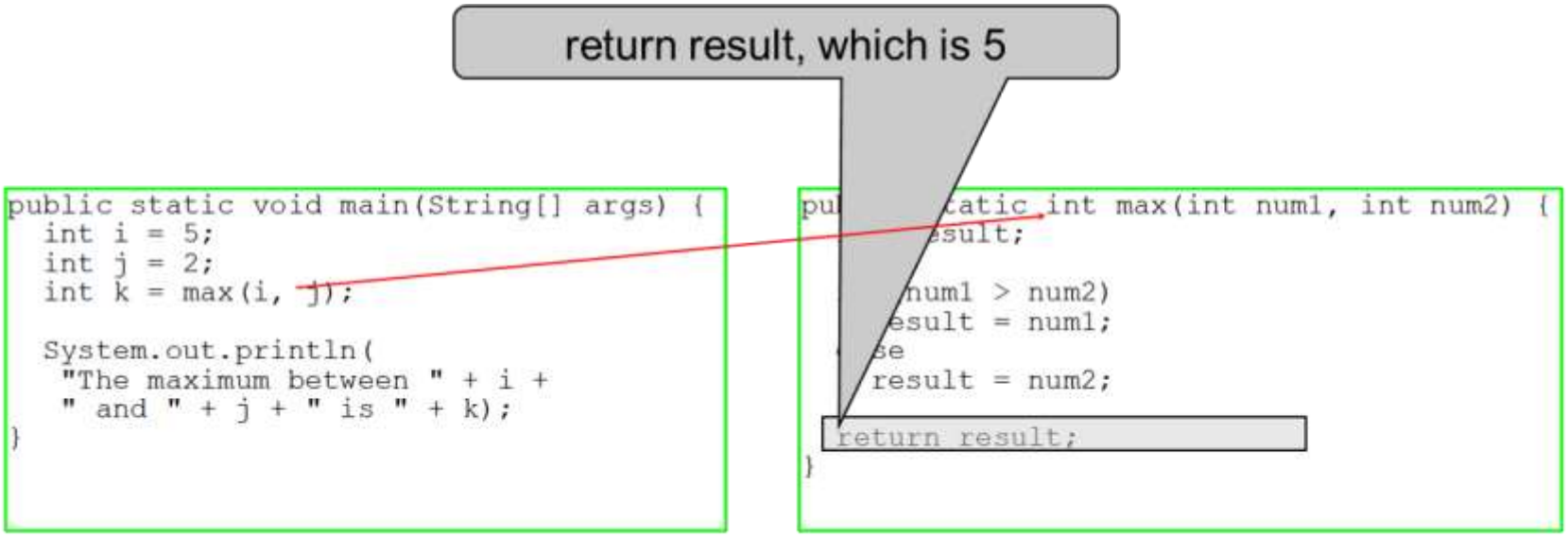
Trace Method Invocation (7 of 10)



Pearson

Copyright © 2024 Pearson Education, Inc. All Rights Reserved

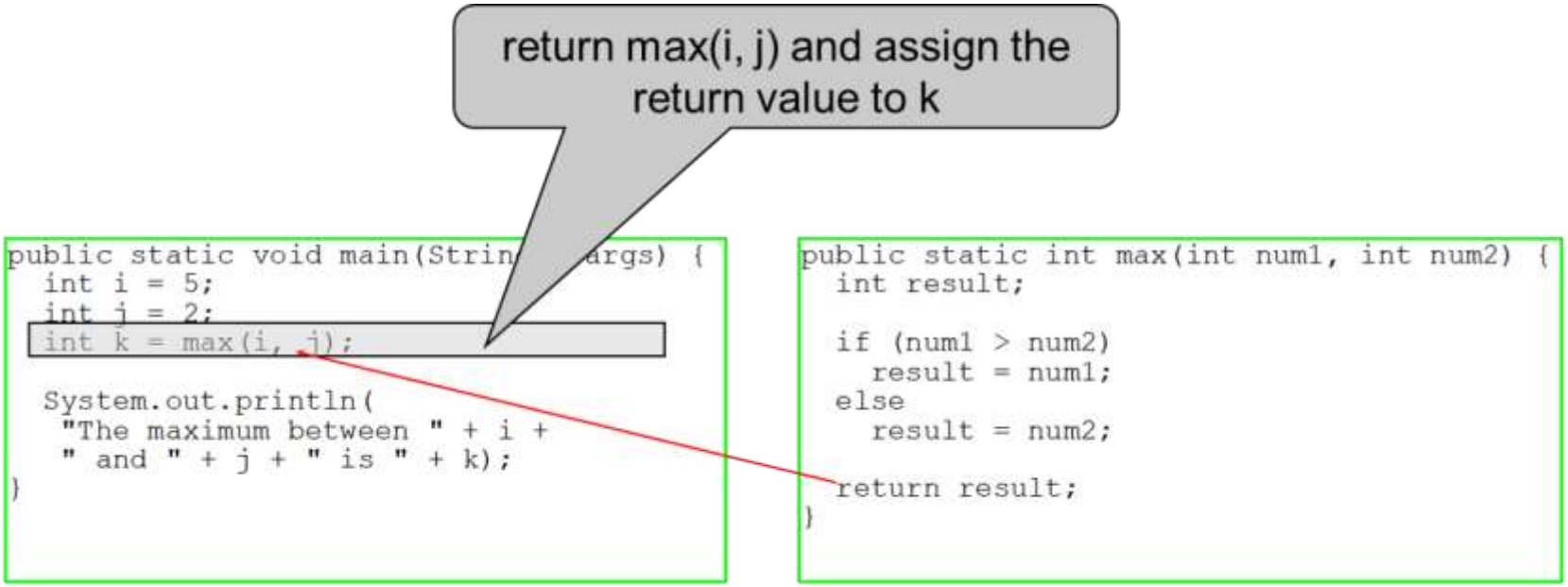
Trace Method Invocation (8 of 10)



Pearson

Copyright © 2024 Pearson Education, Inc. All Rights Reserved

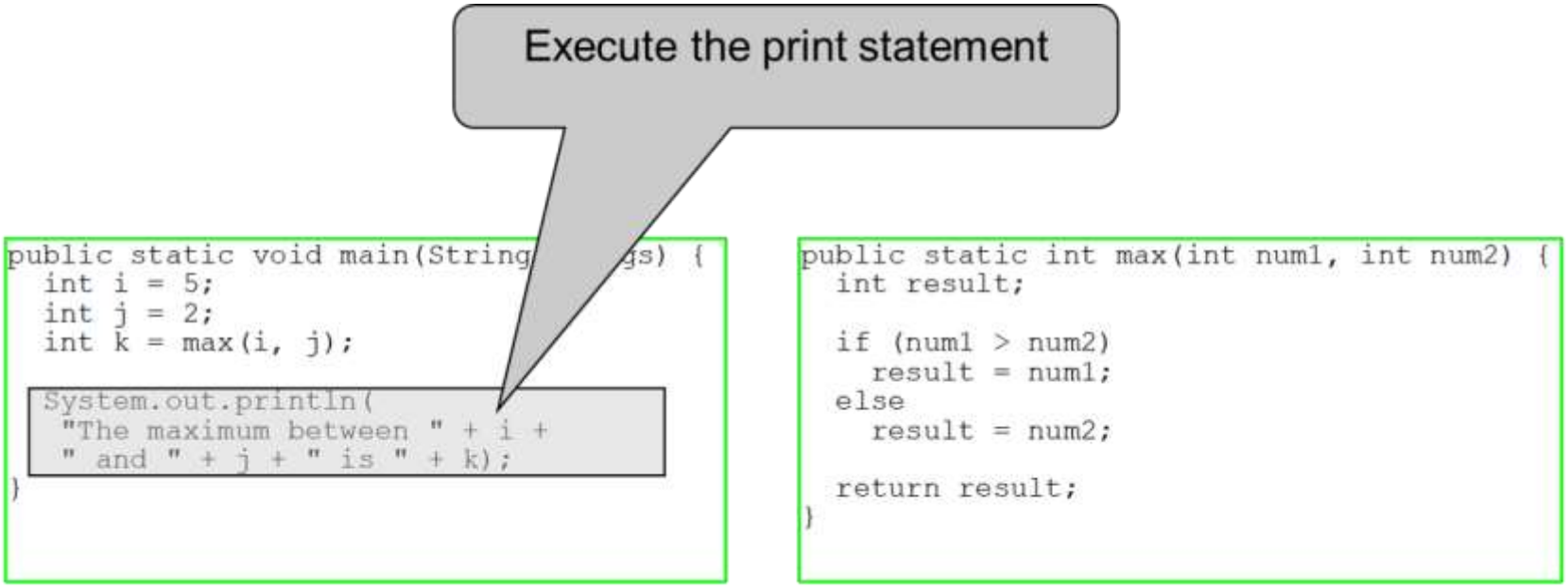
Trace Method Invocation (9 of 10)



Pearson

Copyright © 2024 Pearson Education, Inc. All Rights Reserved

Trace Method Invocation (10 of 10)

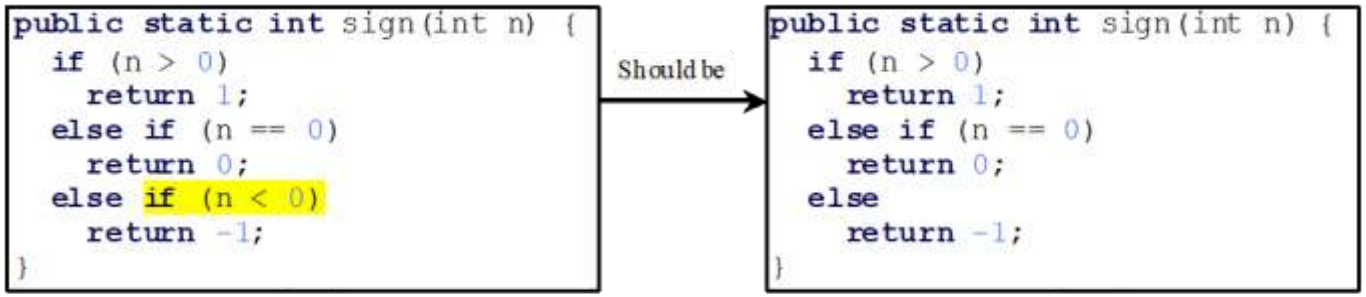


Pearson

Copyright © 2024 Pearson Education, Inc. All Rights Reserved

Caution

A **`return`** statement is required for a value-returning method. The method shown below in (a) is logically correct, but it has a compilation error because the Java compiler thinks it possible that this method does not return any value.



(a) (b)

To fix this problem, delete **`if (n < 0)`** in (a), so that the compiler will see a **`return`** statement to be reached regardless of how the **`if`** statement is evaluated.

Pearson

Copyright © 2024 Pearson Education, Inc. All Rights Reserved

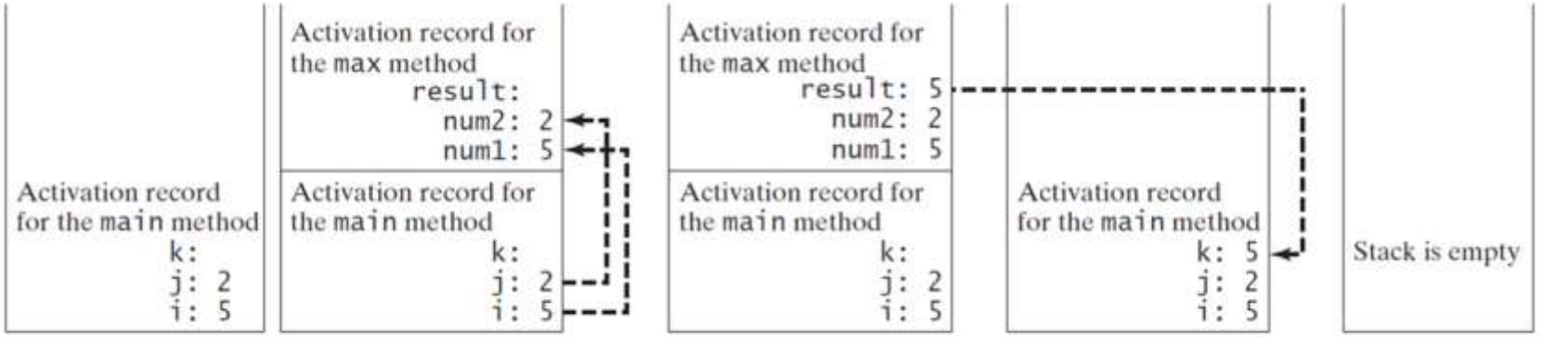
Reuse Methods From Other Classes

Note: One of the benefits of methods is for reuse. The **`max`** method can be invoked from any class besides **`TestMax`**. If you create a new class **`Test`**, you can invoke the **`max`** method using **`ClassName.methodName`** (e.g., **`TestMax.max`**).

Pearson

Copyright © 2024 Pearson Education, Inc. All Rights Reserved

Call Stacks



(a) The `main` method is invoked.

(b) The `max` method is invoked.

(c) The `max` method is being executed.

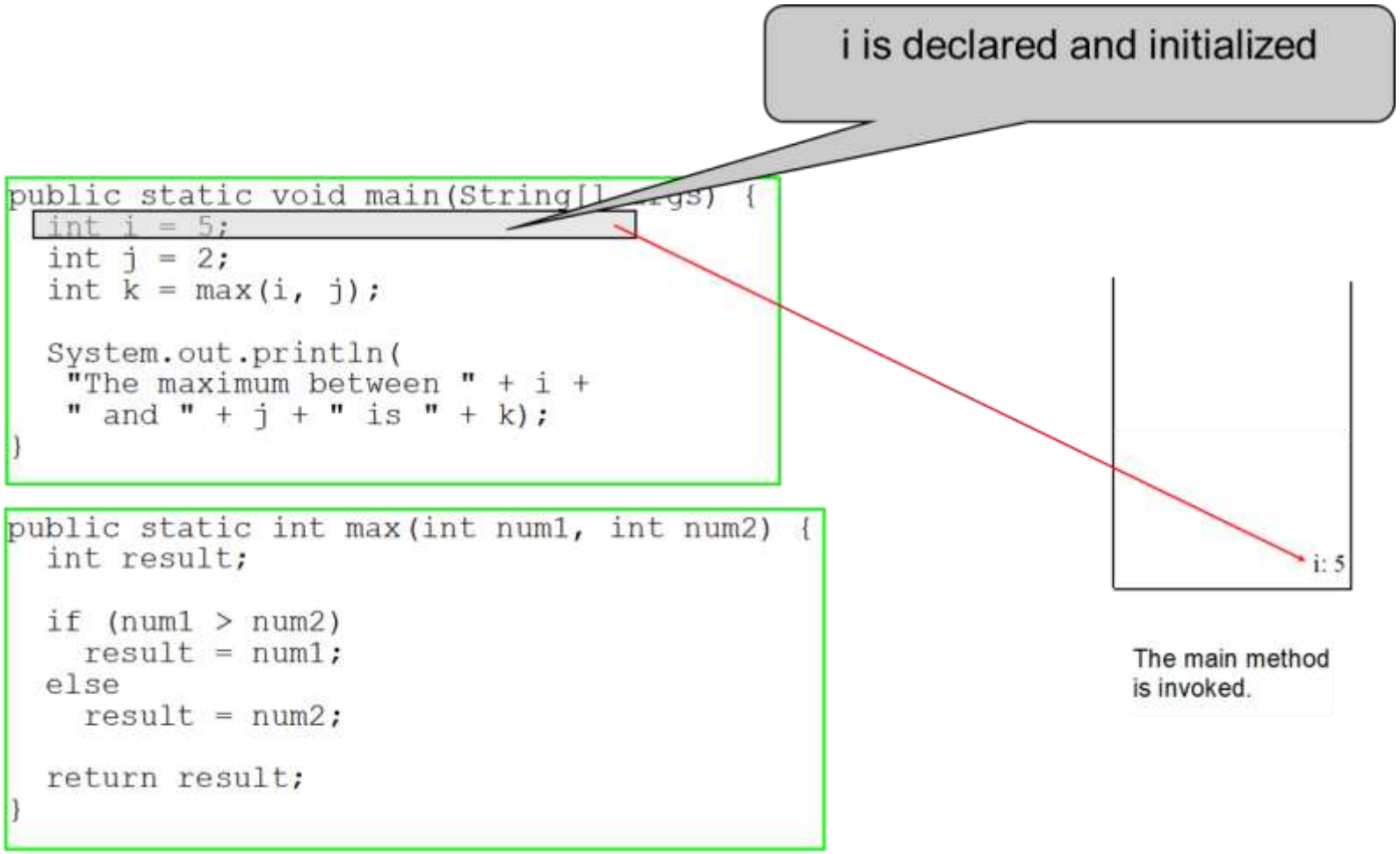
(d) The `max` method is finished and the return value is sent to `k`.

(e) The `main` method is finished.

Pearson

Copyright © 2024 Pearson Education, Inc. All Rights Reserved

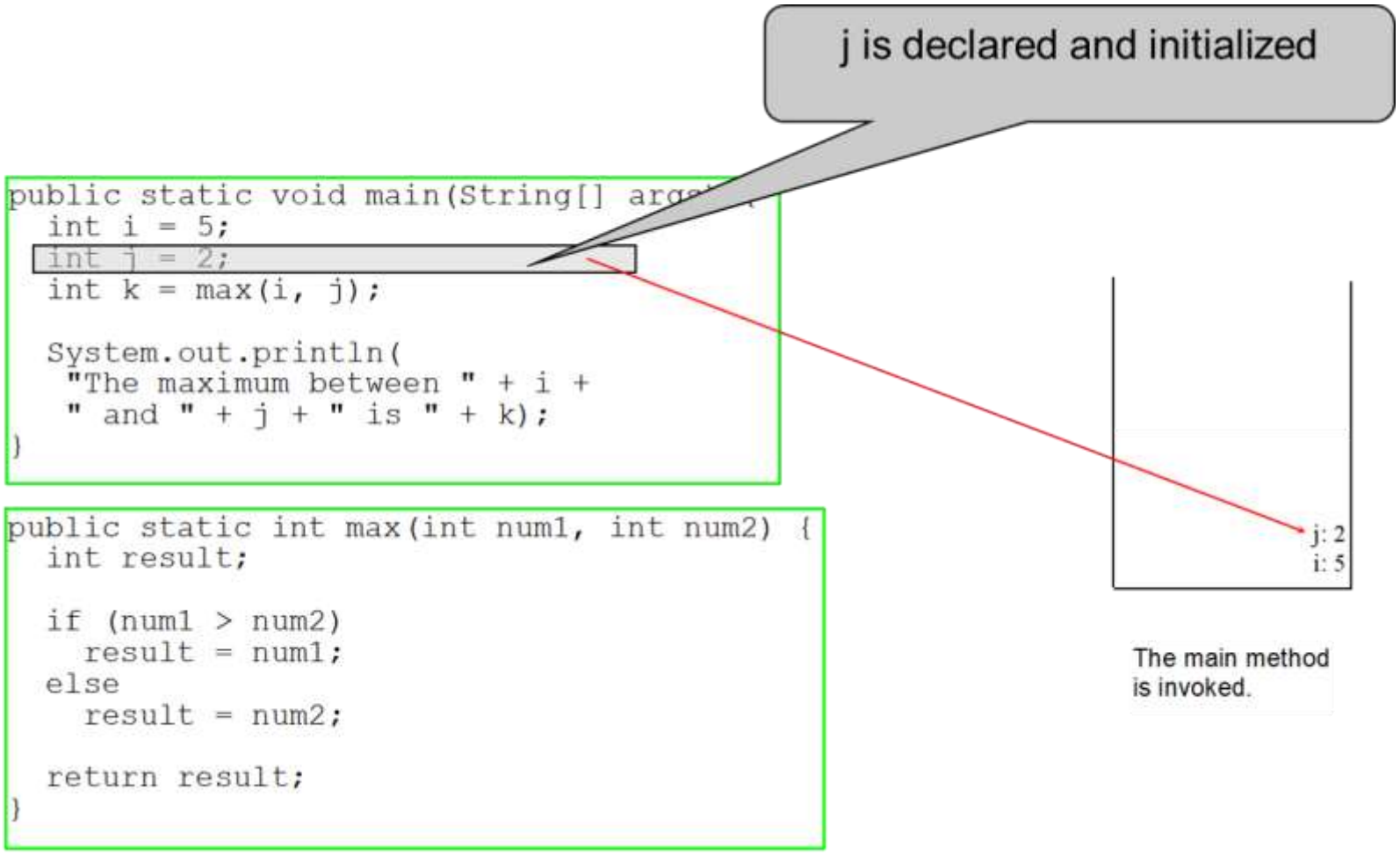
Trace Call Stack (1 of 10)



Pearson

Copyright © 2024 Pearson Education, Inc. All Rights Reserved

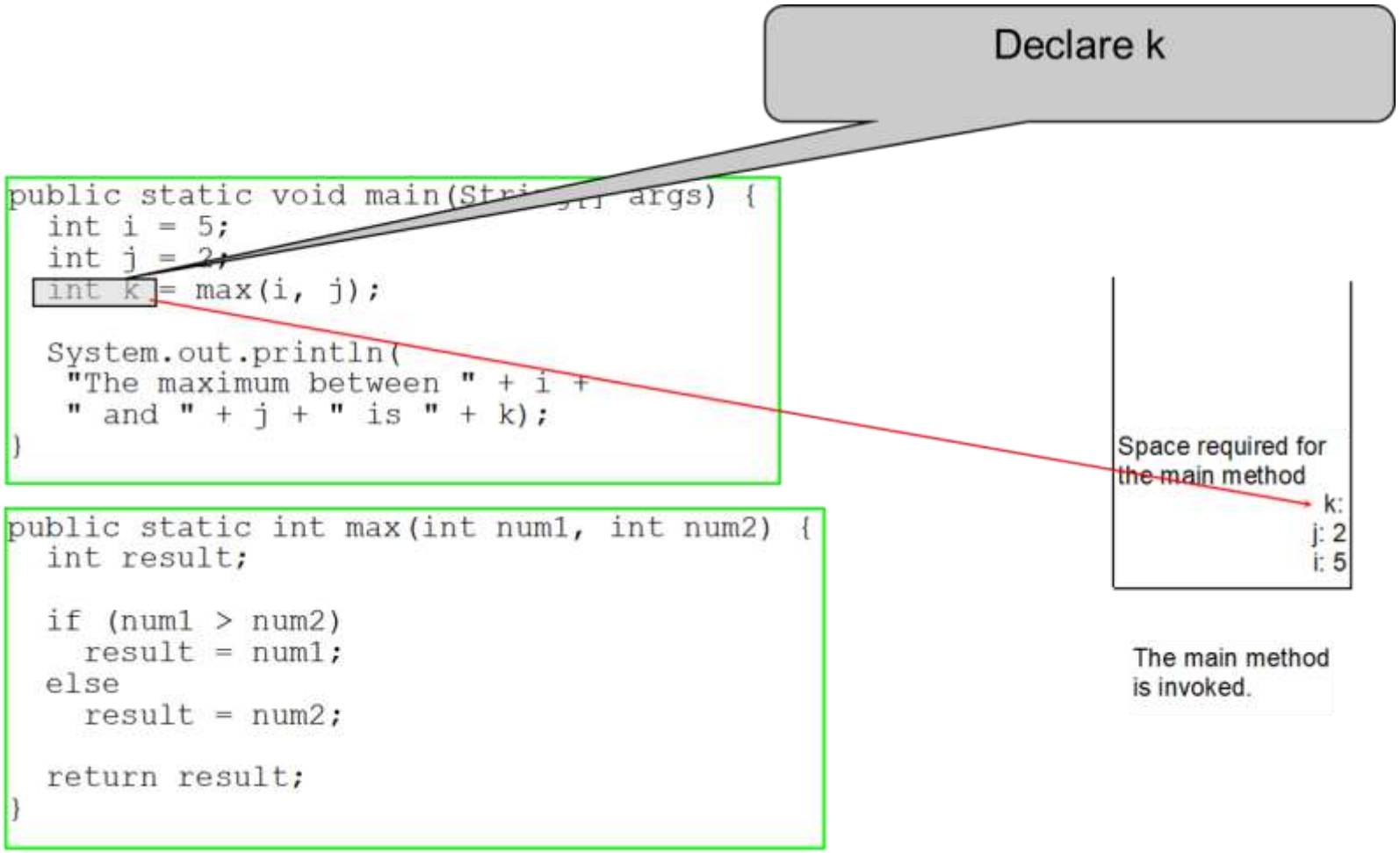
Trace Call Stack (2 of 10)



Pearson

Copyright © 2024 Pearson Education, Inc. All Rights Reserved

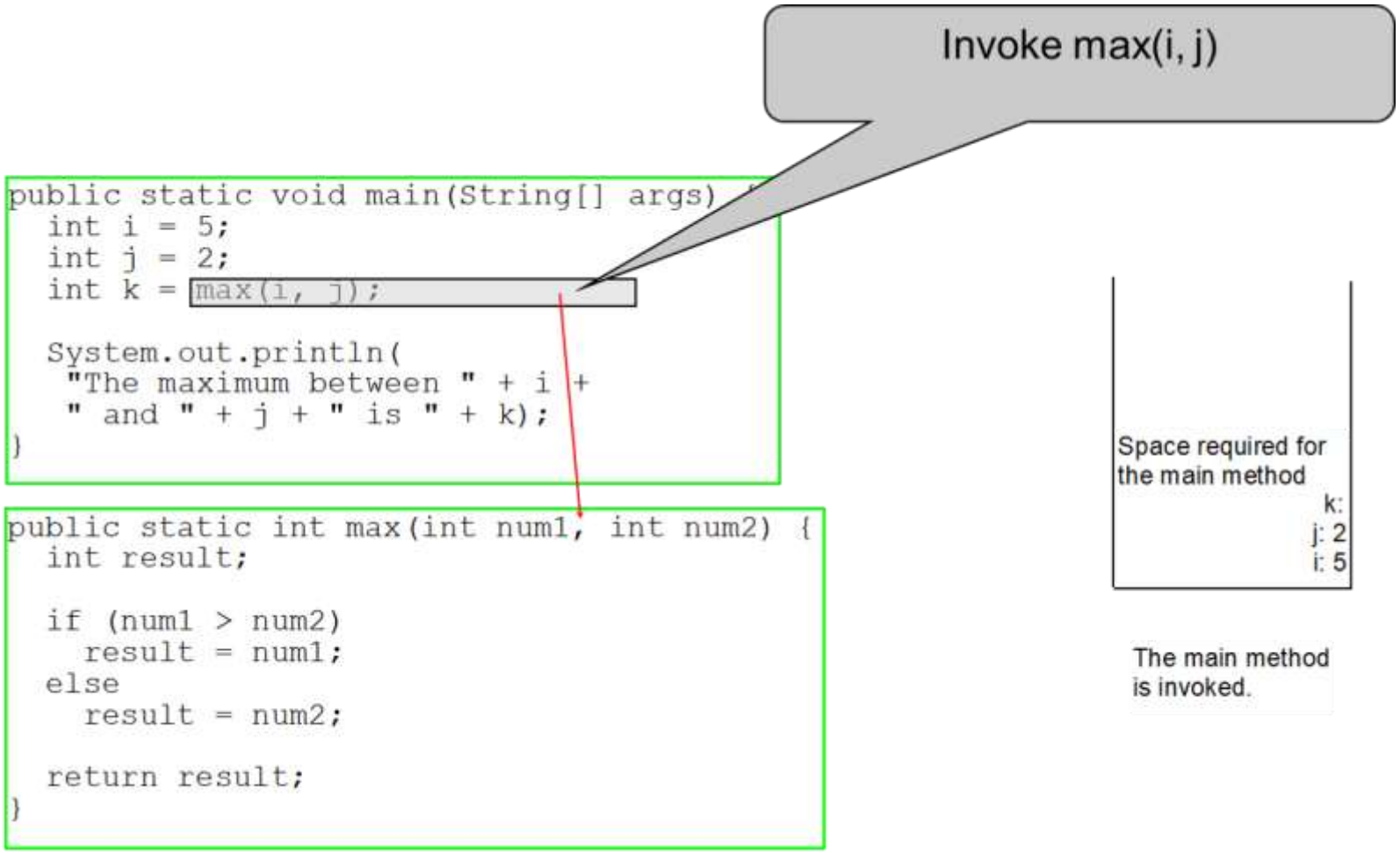
Trace Call Stack (3 of 10)



Pearson

Copyright © 2024 Pearson Education, Inc. All Rights Reserved

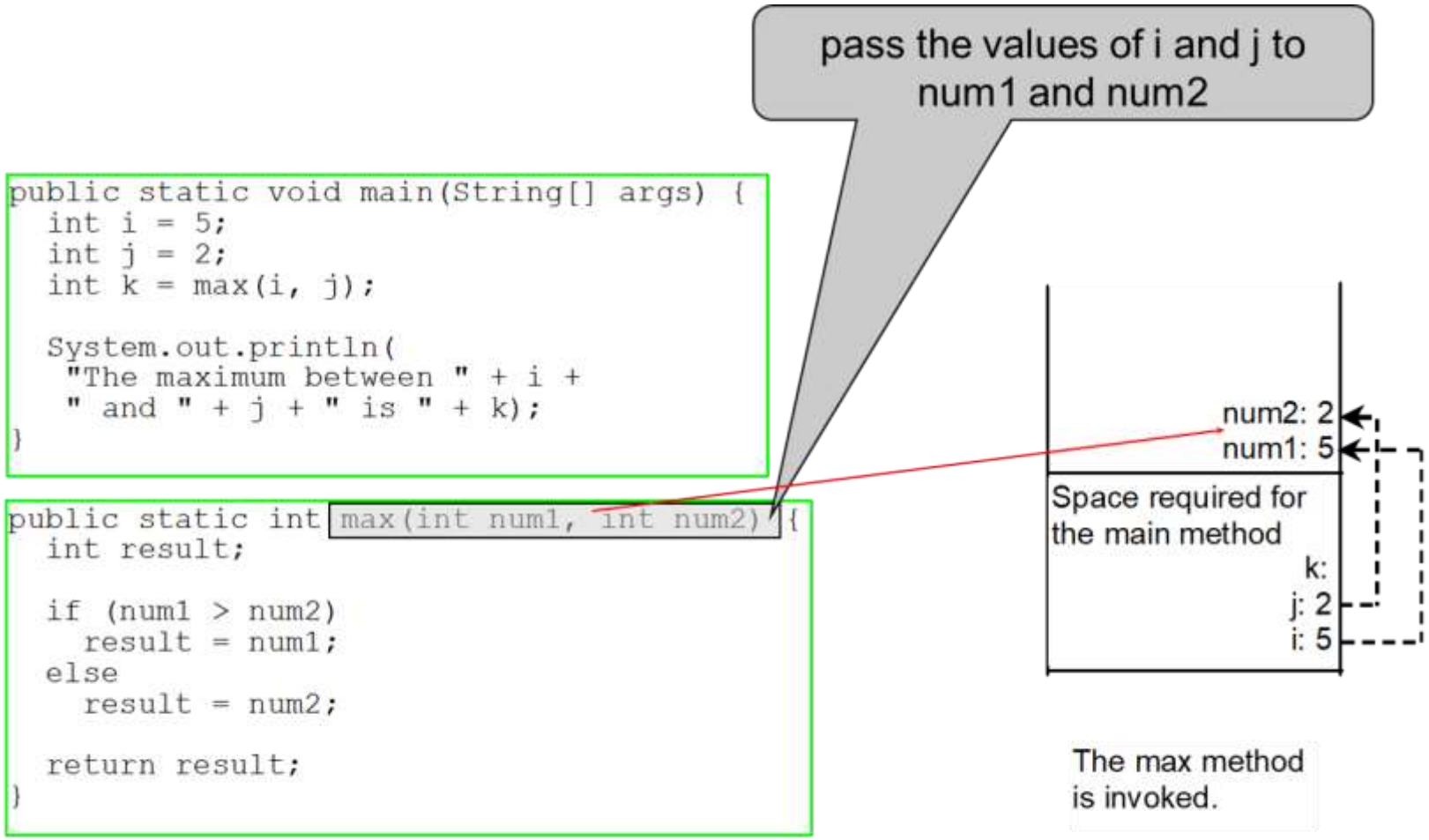
Trace Call Stack (4 of 10)



Pearson

Copyright © 2024 Pearson Education, Inc. All Rights Reserved

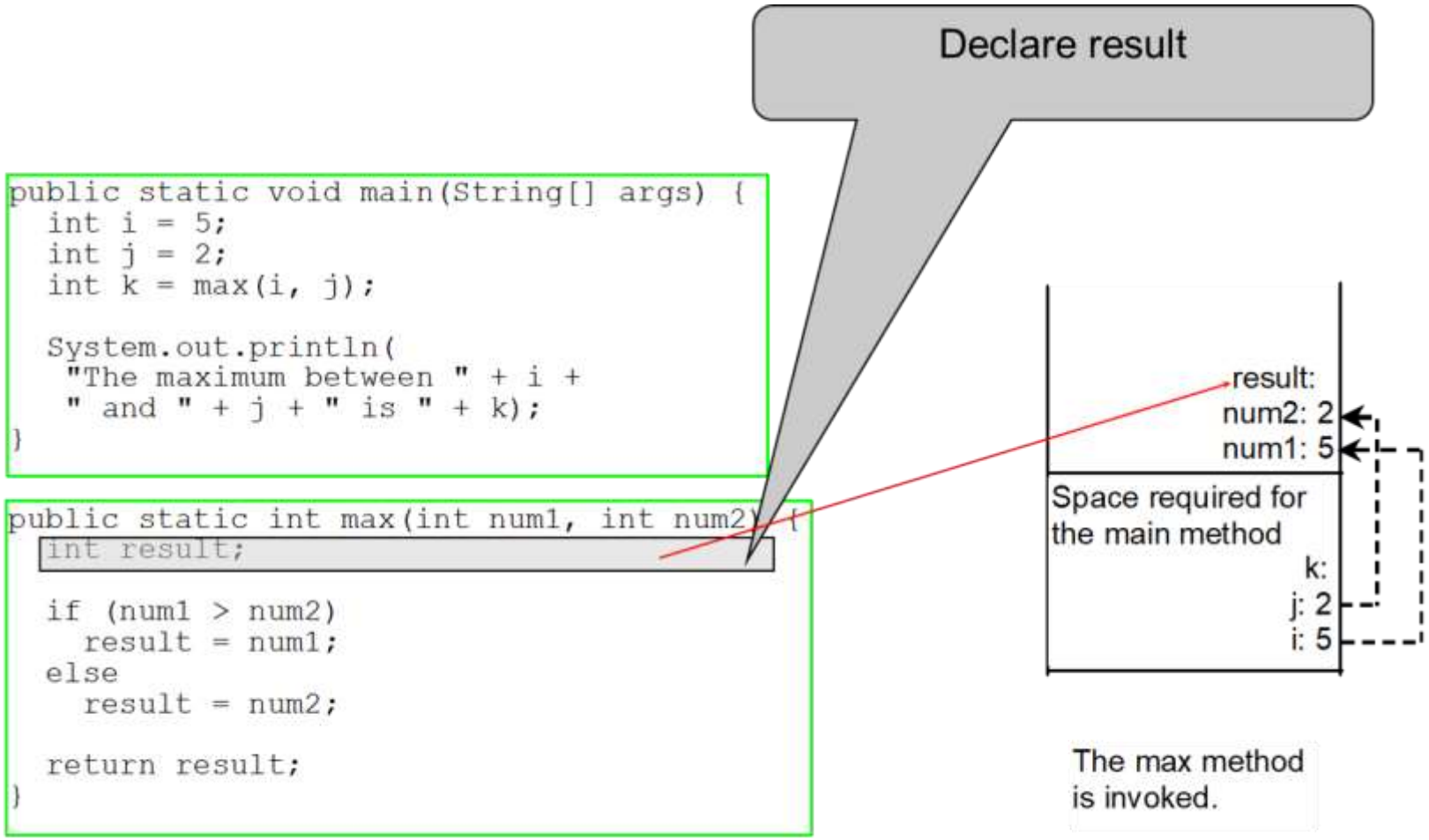
Trace Call Stack (5 of 10)



Pearson

Copyright © 2024 Pearson Education, Inc. All Rights Reserved

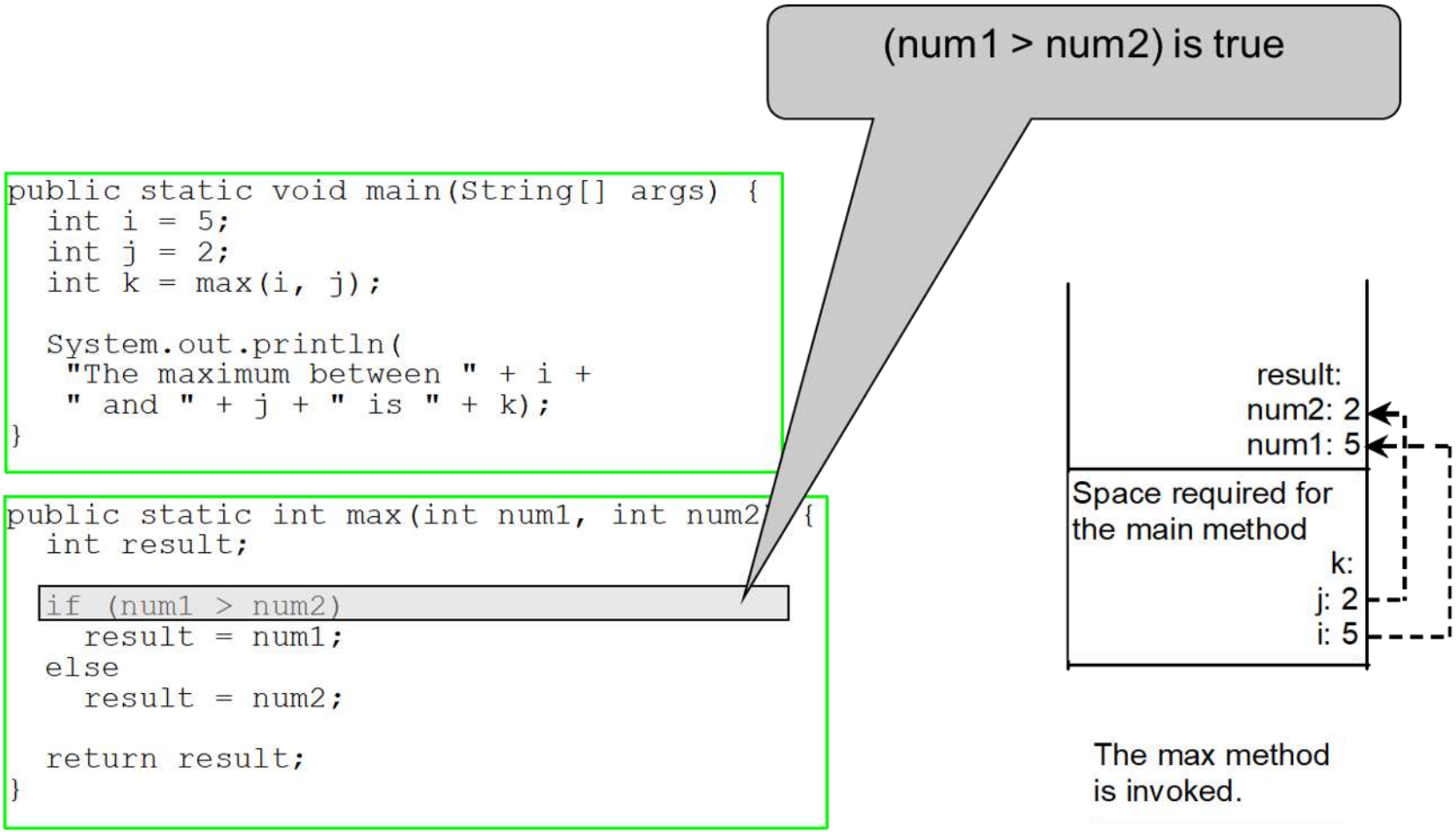
Trace Call Stack (6 of 10)



Pearson

Copyright © 2024 Pearson Education, Inc. All Rights Reserved

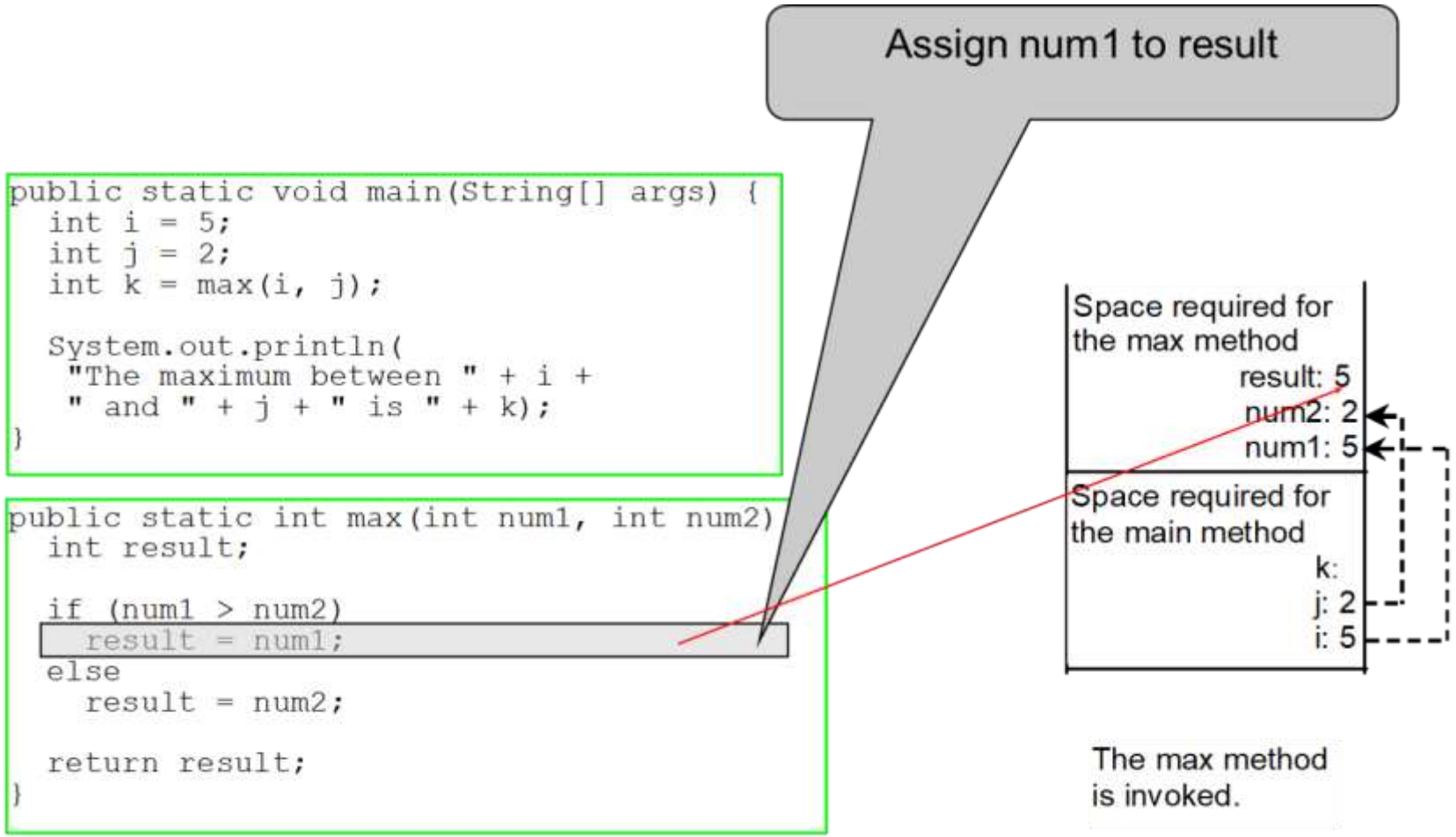
Trace Call Stack (7 of 10)



Pearson

Copyright © 2024 Pearson Education, Inc. All Rights Reserved

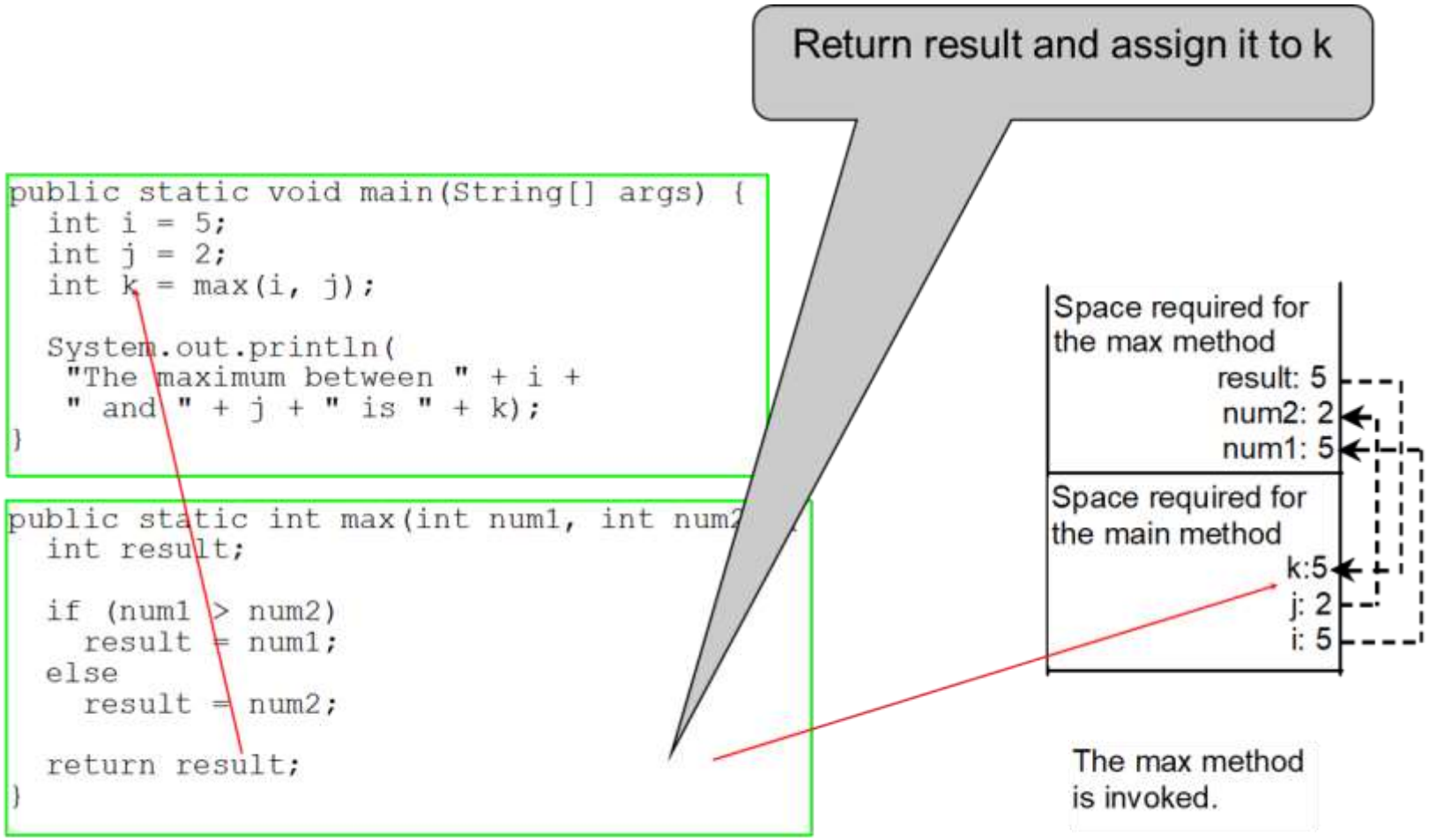
Trace Call Stack (8 of 10)



Pearson

Copyright © 2024 Pearson Education, Inc. All Rights Reserved

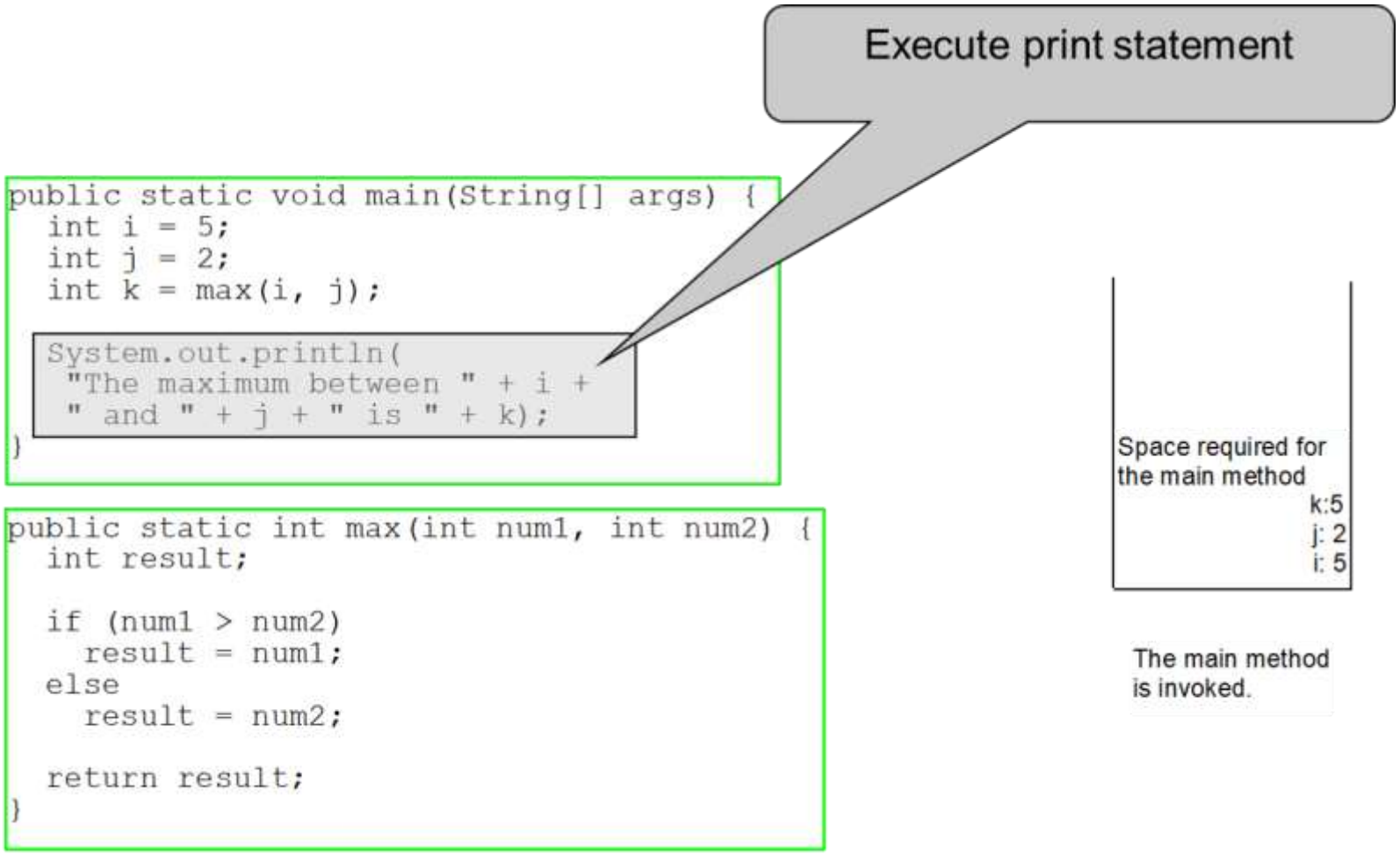
Trace Call Stack (9 of 10)



Pearson

Copyright © 2024 Pearson Education, Inc. All Rights Reserved

Trace Call Stack (10 of 10)



Pearson

Copyright © 2024 Pearson Education, Inc. All Rights Reserved

`void` Method Example

This type of method does not return a value. The method performs some actions.

TestVoidMethod

TestReturnGradeMethod

Pearson

Copyright © 2024 Pearson Education, Inc. All Rights Reserved

Passing Parameters

```
public static void nPrintln(String message, int n) {
   for (int i = 0; i < n; i++)
      System.out.println(message);
}
```

Suppose you invoke the method using

`nPrintln("Welcome to Java", 5);`

What is the output?

Suppose you invoke the method using

`nPrintln("Computer Science", 15);`

What is the output?

Can you invoke the method using

`nPrintln(15, "Computer Science");`

Pearson

Copyright © 2024 Pearson Education, Inc. All Rights Reserved

Pass by Value (1 of 3)

This program demonstrates passing values to the methods.

Increment

Copyright © 2024 Pearson Education, Inc. All Rights Reserved

Pass by Value (2 of 3)

Testing Pass by value

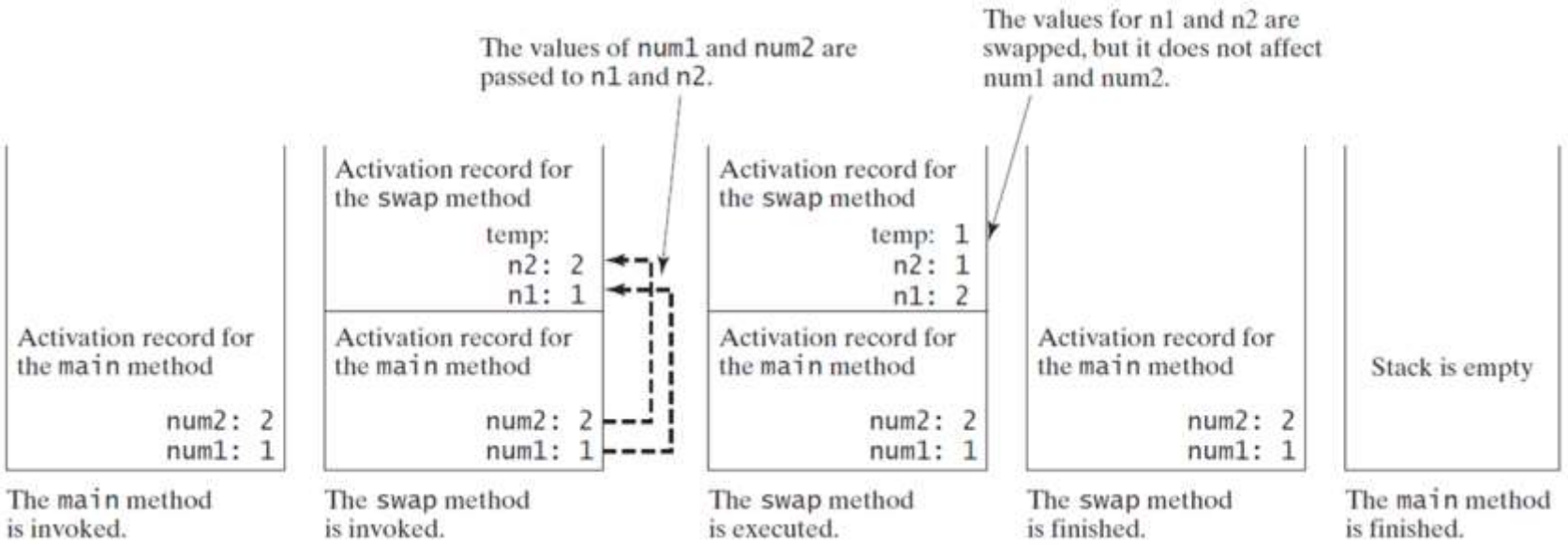
This program demonstrates passing values to the methods.

TestPassByValue

Pearson

Copyright © 2024 Pearson Education, Inc. All Rights Reserved

Pass by Value (3 of 3)



Pearson

Copyright © 2024 Pearson Education, Inc. All Rights Reserved

Modularizing Code

Methods can be used to reduce redundant coding and enable code reuse. Methods can also be used to modularize code and improve the quality of the program.

GreatestCommonDivisorMethod

PrimeNumberMethod

Pearson

Copyright © 2024 Pearson Education, Inc. All Rights Reserved

Case Study: Converting Hexadecimals to Decimals

Write a method that converts a hexadecimal number into a decimal number.

$$ABCD \Rightarrow$$

$$A*16\,\hat{}\,3 + B*16\,\hat{}\,2 + C*16\,\hat{}\,1 + D*16\,\hat{}\,0$$

$$= \left(\left(A*16 + B\right)*16 + C\right)*16 + D$$

$$= \left(\left(10*16 + 11\right)*16 + 12\right)*16 + 13 = ?$$

Hex2Dec

Pearson

Copyright © 2024 Pearson Education, Inc. All Rights Reserved

Overloading Methods

Overloading the `max` Method

```
public static double max(double num1, double
num2) {
  if (num1 > num2)
    return num1;
  else
    return num2;
}
```

TestMethodOverloading

Pearson

Copyright © 2024 Pearson Education, Inc. All Rights Reserved

Ambiguous Invocation (1 of 2)

Sometimes there may be two or more possible matches for an invocation of a method, but the compiler cannot determine the most specific match. This is referred to as **ambiguous invocation**. Ambiguous invocation is a compile error.

Copyright © 2024 Pearson Education, Inc. All Rights Reserved

Ambiguous Invocation (2 of 2)

```
public class AmbiguousOverloading {
  public static void main(String[] args) {
    System.out.println(max(1, 2));
  }
  public static double max(int num1, double num2) {
    if (num1 > num2)
      return num1;
    else
      return num2;
  }
  public static double max(double num1, int num2) {
    if (num1 > num2)
      return num1;
    else
      return num2;
  }
}
```

Pearson

Copyright © 2024 Pearson Education, Inc. All Rights Reserved

Scope of Local Variables (1 of 6)

A local variable: a variable defined inside a method.

Scope: the part of the program where the variable can be referenced.

The scope of a local variable starts from its declaration and continues to the end of the block that contains the variable. A local variable must be declared before it can be used.

Pearson

Copyright © 2024 Pearson Education, Inc. All Rights Reserved

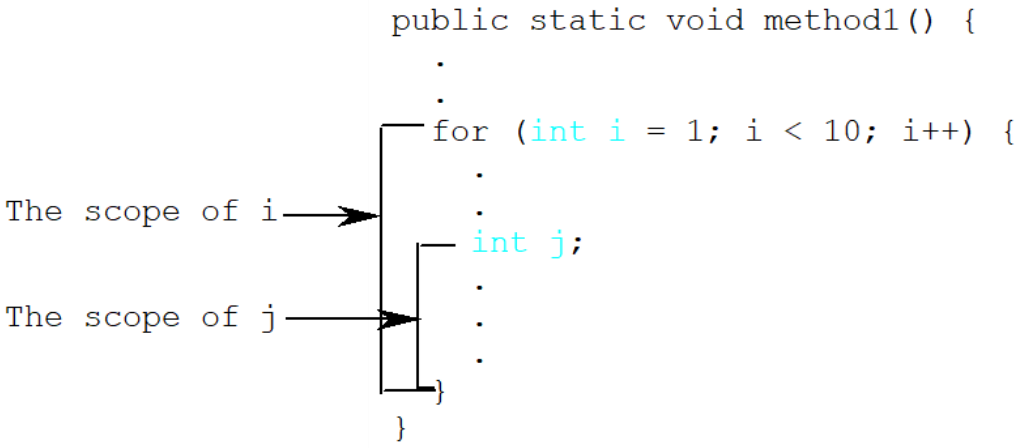
Scope of Local Variables (2 of 6)

You can declare a local variable with the same name multiple times in different non-nesting blocks in a method, but you cannot declare a local variable twice in nested blocks.

Copyright © 2024 Pearson Education, Inc. All Rights Reserved

Scope of Local Variables (3 of 6)

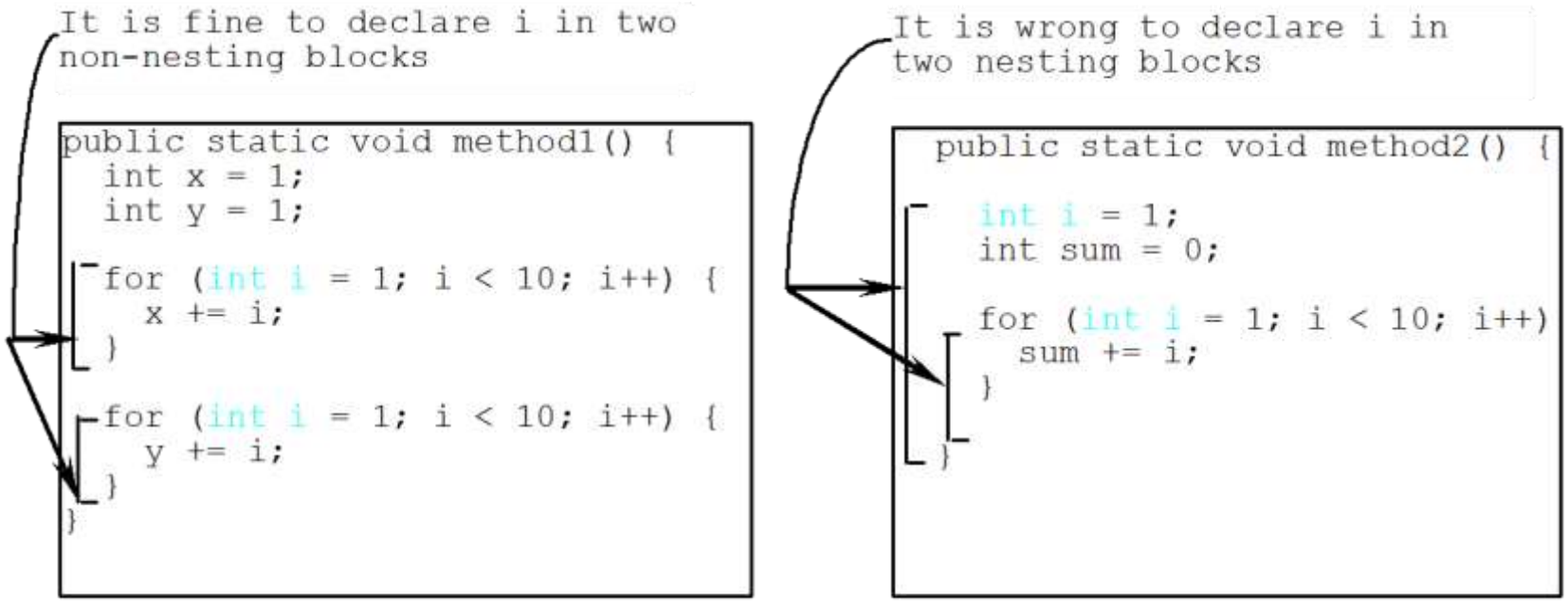
A variable declared in the initial action part of a **`for`** loop header has its scope in the entire loop. But a variable declared inside a **`for`** loop body has its scope limited in the loop body from its declaration and to the end of the block that contains the variable.



Pearson

Copyright © 2024 Pearson Education, Inc. All Rights Reserved

Scope of Local Variables (4 of 6)



Pearson

Copyright © 2024 Pearson Education, Inc. All Rights Reserved

Scope of Local Variables (5 of 6)

```
// Fine with no errors
public static void correctMethod() {
  int x = 1;
  int y = 1;
  // i is declared
  for (int i = 1; i < 10; i++) {
    x += i;
  }
  // i is declared again
  for (int i = 1; i < 10; i++) {
    y += i;
  }
}
```

Pearson

Copyright © 2024 Pearson Education, Inc. All Rights Reserved

Scope of Local Variables (6 of 6)

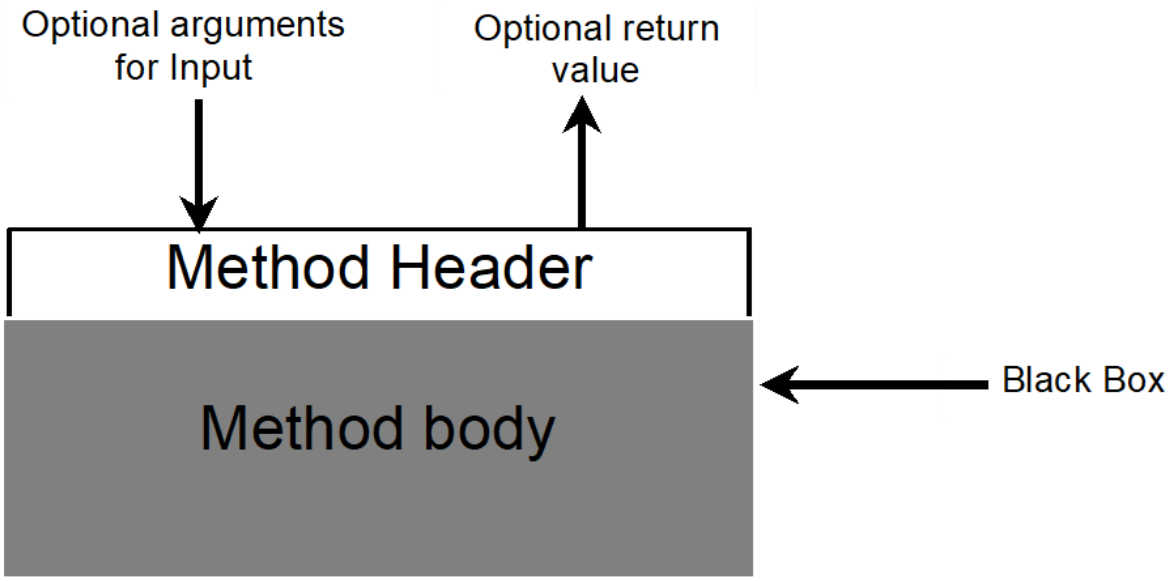
```
// With errors
public static void incorrectMethod() {
   int x = 1;
   int y = 1;
   for (int i = 1; i < 10; i++) {
    int x = 0;
     x += i;
   }
}
```

Pearson

Copyright © 2024 Pearson Education, Inc. All Rights Reserved

Method Abstraction

You can think of the method body as a black box that contains the detailed implementation for the method.



Pearson
Copyright © 2024 Pearson Education, Inc. All Rights Reserved

Benefits of Methods

- Write a method once and reuse it anywhere.
- Information hiding. Hide the implementation from the user.
- Reduce complexity.

Pearson

Copyright © 2024 Pearson Education, Inc. All Rights Reserved

Case Study: Generating Random Characters (1 of 4)

Computer programs process numerical data and characters. You have seen many examples that involve numerical data. It is also important to understand characters and how to process them.

As introduced in Section 4.3, each character has a unique Unicode between 0 and FFFF in hexadecimal (65535 in decimal). To generate a random character is to generate a random integer between 0 and 65535 using the following expression: (note that since `0 <= Math.random() < 1.0`, you have to add 1 to 65535.)

```
(int)(Math.random() * (65535 + 1))
```

Pearson

Copyright © 2024 Pearson Education, Inc. All Rights Reserved

Case Study: Generating Random Characters (2 of 4)

Now let us consider how to generate a random lowercase letter. The Unicode for lowercase letters are consecutive integers starting from the Unicode for `'a'`, then for `'b'`, `'c'`, `...`, and `'z'`. The Unicode for `'a'` is

```
(int)'a'
```

So, a random integer between `(int)'a'` and `(int)'z'` is

```
(int)((int)'a' + Math.random() * ((int)'z' - (int)'a' + 1)
```

Pearson

Copyright © 2024 Pearson Education, Inc. All Rights Reserved

Case Study: Generating Random Characters (3 of 4)

As discussed in Chapter 2, all numeric operators can be applied to the char operands. The char operand is cast into a number if the other operand is a number or a character. So, the preceding expression can be simplified as follows:

```
'a' + Math.random() * ('z' - 'a' + 1)
```

So a random lowercase letter is

```
(char)('a' + Math.random() * ('z' - 'a' + 1))
```

Pearson

Copyright © 2024 Pearson Education, Inc. All Rights Reserved

Case Study: Generating Random Characters (4 of 4)

To generalize the foregoing discussion, a random character between any two characters `ch1` and `ch2` with `ch1 < ch2` can be generated as follows:

```
(char)(ch1 + Math.random() * (ch2 - ch1 + 1))
```

Pearson

Copyright © 2024 Pearson Education, Inc. All Rights Reserved

The RandomCharacter Class (1 of 3)

```
// RandomCharacter.java: Generate random
characters
public class RandomCharacter {
/** Generate a random character between ch1
and ch2 */
  public static char getRandomCharacter(char
  ch1, char ch2) {
     return (char)(ch1 + Math.random() *
     (ch2 - ch1 + 1));
  }

  /** Generate a random lowercase letter */
  public static char
  getRandomLowerCaseLetter() {
     return getRandomCharacter('a', 'z');
  }
```

Pearson

Copyright © 2024 Pearson Education, Inc. All Rights Reserved

The `RandomCharacter` Class (2 of 3)

```
/** Generate a random uppercase letter */
public static char
getRandomUpperCaseLetter() {
   return getRandomCharacter('A', 'Z');
}
```

RandomCharacter

TestRandomCharacter

Pearson

Copyright © 2024 Pearson Education, Inc. All Rights Reserved

The RandomCharacter Class (3 of 3)

```
  /** Generate a random digit character */
  public static char
  getRandomDigitCharacter() {
     return getRandomCharacter('0', '9');
  }

  /** Generate a random character */
  public static char getRandomCharacter() {
     return getRandomCharacter('\u0000',
     '\uFFFF');
  }
}
```

RandomCharacter

TestRandomCharacter

Pearson

Copyright © 2024 Pearson Education, Inc. All Rights Reserved

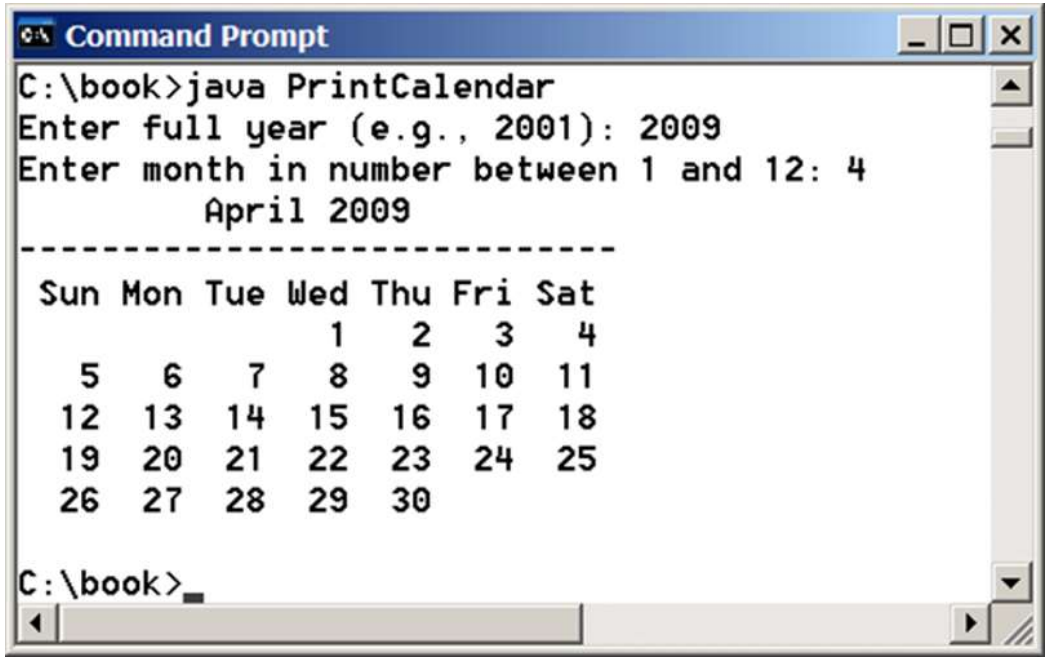
Stepwise Refinement (Optional)

The concept of method abstraction can be applied to the process of developing programs. When writing a large program, you can use the “divide and conquer” strategy, also known as **stepwise refinement**, to decompose it into subproblems. The subproblems can be further decomposed into smaller, more manageable problems.

Copyright © 2024 Pearson Education, Inc. All Rights Reserved

`PrintCalender` Case Study

Let us use the `PrintCalendar` example to demonstrate the stepwise refinement approach.

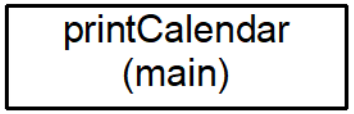


PrintCalendar

Pearson

Copyright © 2024 Pearson Education, Inc. All Rights Reserved

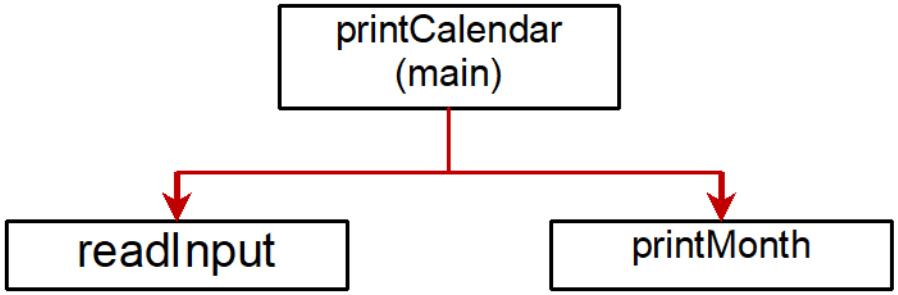
Design Diagram (1 of 7)



Pearson

Copyright © 2024 Pearson Education, Inc. All Rights Reserved

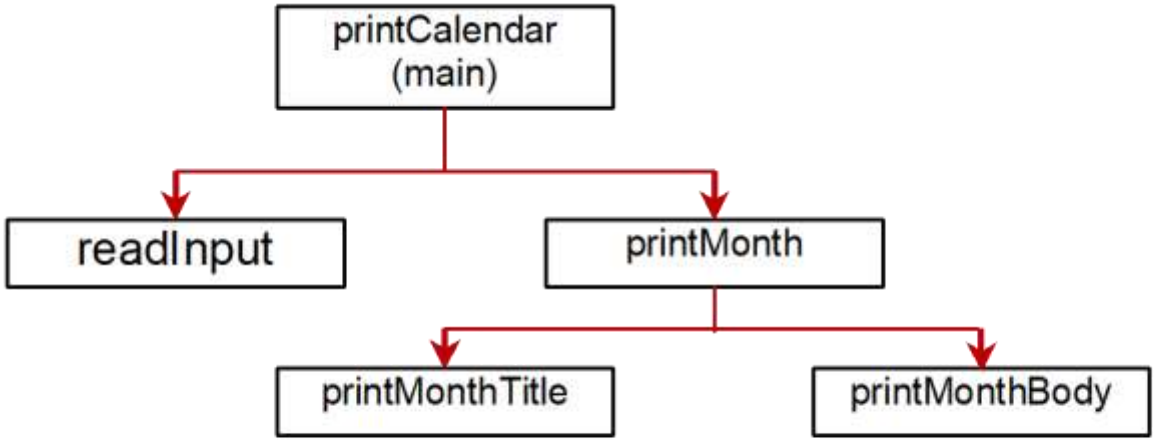
Design Diagram (2 of 7)



Pearson

Copyright © 2024 Pearson Education, Inc. All Rights Reserved

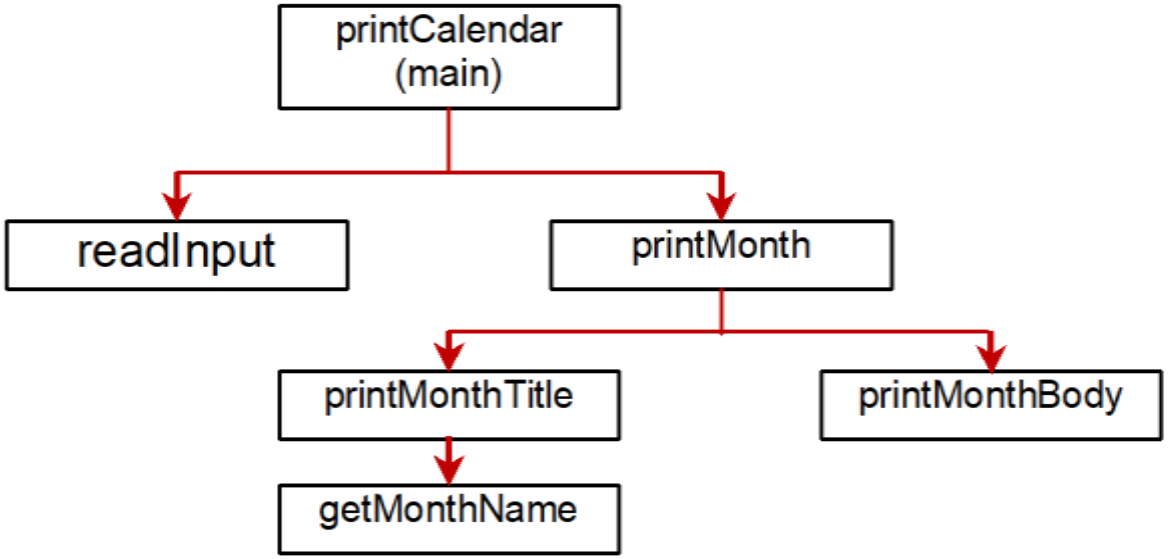
Design Diagram (3 of 7)



Pearson

Copyright © 2024 Pearson Education, Inc. All Rights Reserved

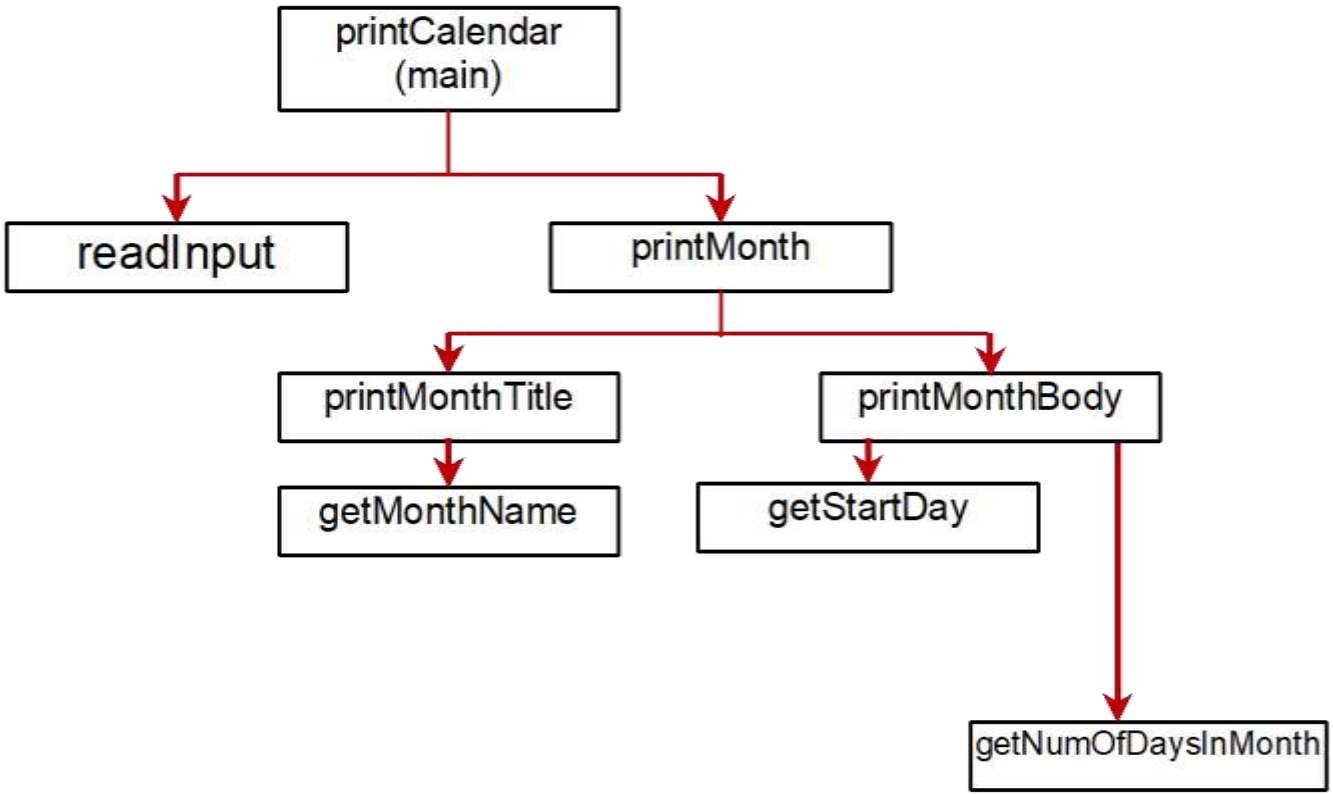
Design Diagram (4 of 7)



Pearson

Copyright © 2024 Pearson Education, Inc. All Rights Reserved

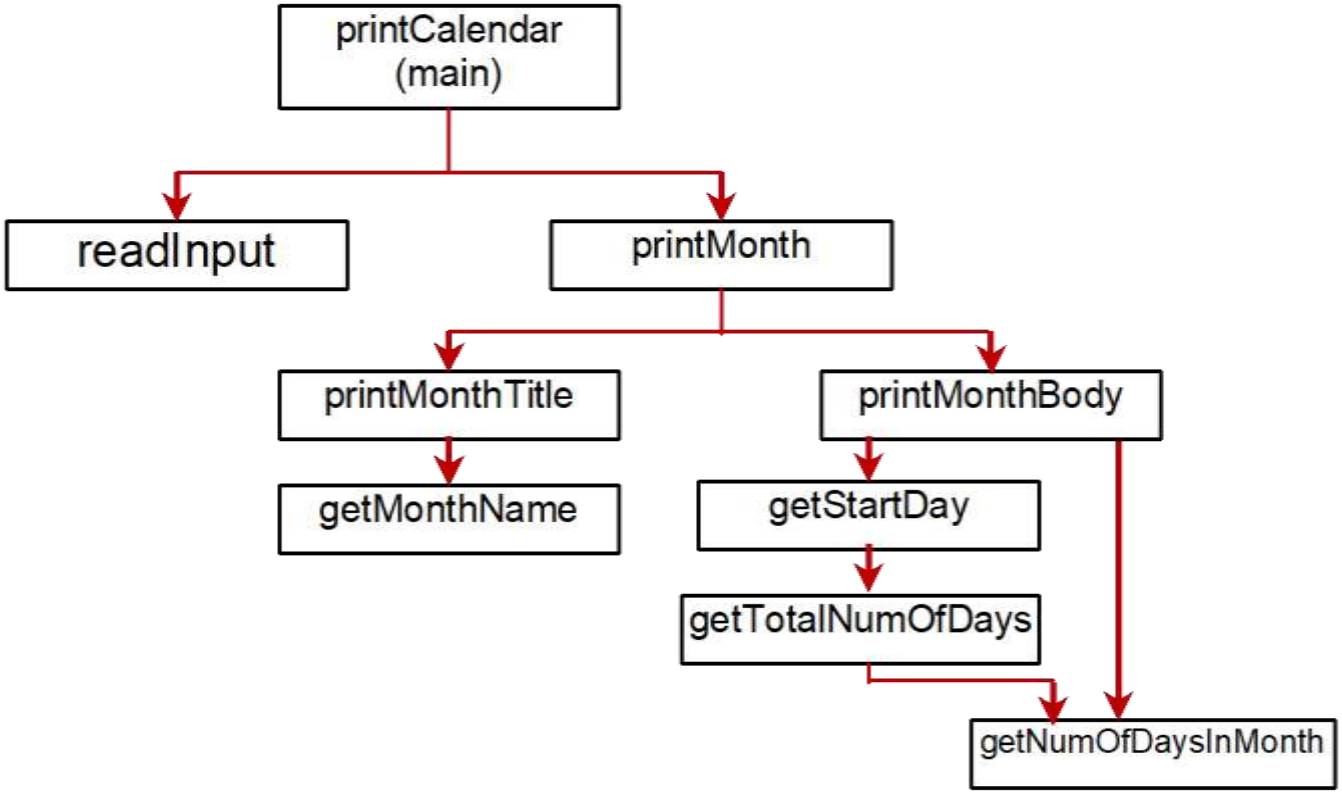
Design Diagram (5 of 7)



Pearson

Copyright © 2024 Pearson Education, Inc. All Rights Reserved

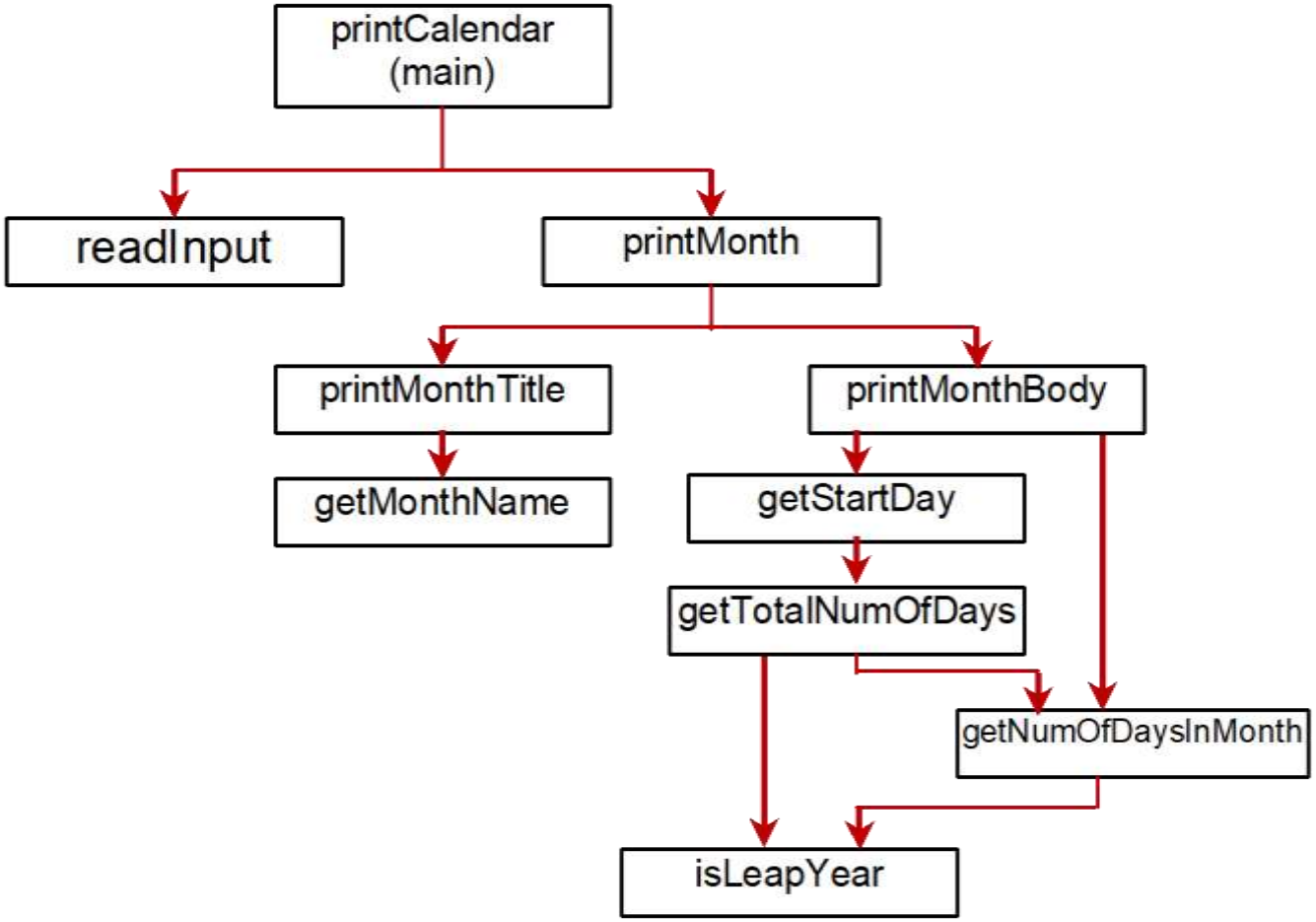
Design Diagram (6 of 7)



Pearson

Copyright © 2024 Pearson Education, Inc. All Rights Reserved

Design Diagram (7 of 7)



Pearson

Copyright © 2024 Pearson Education, Inc. All Rights Reserved

Implementation: Top-Down

Top-down approach is to implement one method in the structure chart at a time from the top to the bottom. Stubs can be used for the methods waiting to be implemented. A stub is a simple but incomplete version of a method. The use of stubs enables you to test invoking the method from a caller. Implement the main method first and then use a stub for the printMonth method. For example, let printMonth display the year and the month in the stub. Thus, your program may begin like this:

A Skeleton for printCalendar

Pearson

Copyright © 2024 Pearson Education, Inc. All Rights Reserved

Implementation: Bottom-Up

Bottom-up approach is to implement one method in the structure chart at a time from the bottom to the top. For each method implemented, write a test program to test it. Both top-down and bottom-up methods are fine. Both approaches implement the methods incrementally and help to isolate programming errors and makes debugging easy. Sometimes, they can be used together.

Copyright © 2024 Pearson Education, Inc. All Rights Reserved

Benefits of Stepwise Refinement

Simpler Program

Reusing Methods

Easier Developing, Debugging, and Testing

Better Facilitating Teamwork

Pearson

Copyright © 2024 Pearson Education, Inc. All Rights Reserved

Copyright



This work is protected by United States copyright laws and is provided solely for the use of instructors in teaching their courses and assessing student learning. Dissemination or sale of any part of this work (including on the World Wide Web) will destroy the integrity of the work and is not permitted. The work and materials from it should never be made available to students except by instructors using the accompanying text in their classes. All recipients of this work are expected to abide by these restrictions and to honor the intended pedagogical purposes and the needs of other instructors who rely on these materials.

Pearson

Copyright © 2024 Pearson Education, Inc. All Rights Reserved